Constitutional Consensus for Democratic Governance

Anonymous author
Anonymous affiliation

Abstract

Permissionless consensus provides the strata for Decentralised Autonomous Organisations (DAOs) — the prevailing paradigm for participant-governed digital organisations. However, as participants have verified resources but no trusted identities, this ecosystem is necessarily plutocratic (one coin – one vote). Here we present Constitutional Consensus, a Byzantine-fault-tolerant consensus protocol for the democratic (one person – one vote) governance of digital communities and organisations through self-governing amendment of the protocol's own parameters.

Echoing Lamport's vision of consensus as a self-governing parliament, the protocol operates in epochs, each governed by a prevailing constitution that specifies the amendable parameters — the participant set, the supermajority threshold, and the message-delay bound — and the democratic decision processes by which they are amended. An epoch ends with the ratification of a constitutionally-valid amendment, upon which the next epoch begins. Constitutional amendment generalises reconfiguration of consensus protocols in two respects: in *scope*, from the participant set to any constitutional parameter; and in *authority*, from a designated leader or external entity to the participants themselves, deciding through processes specified by the constitution.

The single-epoch consensus protocol is DAG-based, following Cordial Miners thus eschewing reliable broadcast, and following Morpheus dual-modes, thus quiescent when idle. It achieves consensus in 3δ, maintains $O(n)$ amortized communication complexity during high throughput, and seamlessly transitions between modes. The basic protocol (without constitutional amendments) realizes these features in 25 lines of pseudocode, making it one of the most concise consensus protocols for eventual synchrony.

2012 ACM Subject Classification Computer systems organization → Peer-to-peer architectures; Software and its engineering → Distributed systems organizing principles

Keywords and phrases Consensus, Paxos, constitutional amendment, digital social contracts, democratic digital communities, grassroots

This paper is not eligible for the best student paper award.

1 Introduction

Participant-governance of digital organisations. Permissionless-consensus-based Decentralised Autonomous Organisations (DAOs) [29, 27, 35, 75] are the prevailing paradigm for realising participant-governed digital organisations. As participants have verified resources (e.g., staked tokens) but no trusted identities, this ecosystem is necessarily plutocratic: voting power is proportional to token holdings (one coin – one vote). Here we offer, for the first time, a democratic (one person – one vote) paradigm for the governance of digital communities and organisations, based on permissioned consensus and egalitarian decision processes.

Constitutional self-governance. Reconfiguration—changing the set of participants in a consensus protocol—has been studied extensively [43, 44, 45, 2, 74, 56]. This literature addresses the mechanism of *how* membership changes execute safely, but not *who decides* on them: the decision authority is either left unspecified or delegated to an external entity. Moreover, prior work applies reconfiguration only to membership, leaving the rest of the protocol fixed.[1] In line with Lamport's vision of consensus as a self-governing parliament [43], we propose a democratic paradigm in which a *constitution* specifies both a decision-making protocol and a consensus protocol, combined to let participants amend the constitution through constitutionally-valid decisions that are ratified by consensus. The protocol operates in epochs, each governed by a prevailing constitution; an epoch ends with the ratification of a valid amendment, upon which the next begins.

Digital social contracts. A digital social contract [20] is a voluntary agreement among a set of people, specified, fulfilled, and enforced digitally. Many digital social contracts — including social graphs [70, 51], social networks [69], and digital currencies [67, 49]—require no consensus at all. The goal of this work is a consensus protocol that is simple and efficient enough so that social contracts that do require consensus among participants—notably democratic governance and federation of sovereign communities [76, 72, 34]—could also be executed on the participants' own devices.

Constitutional Consensus is designed for such social contracts. Unlike smart contracts [26], whose execution is enforced by global consensus among third-party validators on a blockchain, digital social contracts are among people with mutually-known identities and are enforced by the participants themselves. This distinction—summarised in Table 1 in Appendix G—has a direct consequence: governance is democratic rather than plutocratic. The democratic decision processes for such governance are developed in constitutional governance in metric spaces [72], which may run over Constitutional Consensus; Section 4 recalls the rules employed here.

Technical approach. The single-epoch consensus protocol is DAG-based, following Cordial Miners [40] thus eschewing reliable broadcast, with dual-mode operation following Morpheus [50]: quiescent when idle, with spontaneous leaders for isolated transactions and formal round-robin leadership during high throughput. Neither protocol combines both properties: Cordial Miners is not quiescent, and Morpheus orders via protocol messages outside its DAG. Constitutional Consensus embeds every protocol message except the two dissemination-inducing blocks in the blocklace and is quiescent when idle. Quiescence is required by the intended application—digital communities whose members' devices cannot sustain perpetual block generation and whose transactions may be hours or days apart; embedding the protocol in the blocklace keeps it simple and lets an epoch's blocklace carry

[1] Tezos [33] has no notion of membership but can self-modify the protocol via plutocratic governance.

over intact to the next under constitutional amendment. After GST, a leader block is finalised within 3δ, and the amortized communication complexity per transaction is $O(n)$ during high throughput. The single-epoch protocol is quite simple, requiring only 25 lines of pseudocode.

Grassroots. Because Constitutional Consensus requires no external authority, global ledger, or staking globally-controlled capital, the multi-epoch protocol is *grassroots* [68, 70, 51]: each member can initiate and participate in multiple independent instances that can form, operate and grow concurrently without coordination or dependence on global resources other than the net, and may coalesce when interconnected. This is in contrast with blockchain protocols [55, 18], whose shared global ledger precludes multiple independent instances. More generally, any protocol that employs a shared global data structure—whether replicated (Blockchain [55]) or distributed (DHT [61], IPFS [7])—is not oblivious, and hence not grassroots [51].

Paper structure. Section 2 provides the model and problem definitions. Section 3 presents single-epoch Constitutional Consensus, with no constitutional amendments. Section 4 presents Sybil-resilient democratic decision processes for amending the set of participants, supermajority threshold, and timeout. Section 5 presents the formal amendment mechanism and epoch transitions. Section 6 summarises related work, with a full survey in Appendix G. Section 7 concludes. Correctness proofs and other technicalities are deferred to appendices.

2 Model and Problem

We assume a set of *agents* Π, each has a unique key-pair and identified by its public key $p \in \Pi$. While Π is potentially-infinite, we refer only to finite subsets $P \subset \Pi$ of it. A *correct* agent follows the protocol. An agent that does not follow the protocol is *faulty* or *Byzantine*.

We assume an *eventual synchrony* (*partial synchrony)* model in which every message sent between correct agents is eventually received (this abstracts a dissemination layer that can use acks, gossip, or network coding, and is orthogonal to the consensus protocol). Message latency is unbounded, but in every run there is a time called the *Global Stabilization Time (GST)*, which is unknown to the agents, after which all messages arrive within δ time for some $\delta > 0$.

In general, the constitution specifies the entire set of protocols employed—decision making processes as well as the consensus protocol for ratifying them. In our case, these consist of two protocols: the known democratic decision processes recalled in Section 4 and the novel Constitutional Consensus protocol, which is the main subject of this paper. In this work the constitution assumes both of these protocols to be fixed except for three amendable components, the participant set, the supermajority threshold, and the timeout parameter.

A protocol proceeds through a sequence of *epochs*, each commences with a new constitution, referred to as the *prevailing constitution* during the epoch, and ends when the constitution is *amended*, namely replaced with a new constitution:

▶ **Definition 1** (Amendable component of the Constitution). *The* amendable component of the constitution *is the triple* (P, σ, Δ) *where:*

1. $P \subset \Pi$ *is a set of agents, referred to as its* participants*, with* $n := |P|$*,*
2. $\frac{1}{2} \leq \sigma < 1$ *is a fraction that specifies a supermajority among the participants,*
3. $\Delta > 0$ *is a presumed upper-bound on message delay after GST among the participants.*

We may omit Δ *when discussing protocol safety and refer to the constitution* (P, σ)*. A* σ-supermajority *among the participants is a subset* $Q \subseteq P$ *such that* $|Q| > \sigma n$*.*

In the following we may use *the constitution* as a shorthand, e.g. "the prevailing constitution" instead of "the amendable component of the prevailing constitution".

Agents vs Participants. Throughout this paper, we distinguish between *agents* and *participants*: Any member of Π is an *agent*, while members of the set P in a constitution (P, σ, Δ) are *participants* in the epoch with this prevailing constitution. Only participants execute the protocol during an epoch, though any agent may become a participant through constitutional amendments.

Supermajorities, Sybils and Byzantines. Our protocols are defined using a supermajority that aims to overcome participants that are (1) Byzantine (arbitrarily-behaving) [47] and (2) Sybil (with fake or duplicate identity) [66].

Byzantines and Sybils are related (e.g. Byzantines may aim to create and control multiple Sybils to amplify their ability to cause harm) but not the same: Byzantine behaviour relates to machines deviating from the protocol, whereas Sybils relate to a single person controlling multiple accounts (or personal machines). The volitional framework [51] formalises this distinction, modelling an agent as a person operating a machine. It separates how Byzantines and Sybils may hamper the two protocols: in consensus, the machines order blocks, and this order may be sabotaged by Byzantine machines; democratic decisions are cast as votes, which are volitional choices of persons, so they are hampered by Sybil persons casting multiple votes.

A realisation of Constitutional Consensus may incorporate a different supermajority for consensus and for democratic decisions. For simplicity we assume an upper bound $f < n$ on the number of faulty (Byzantine and Sybil) participants, and define $\sigma := \frac{n+f}{2n}$; equivalently, σ is chosen under the assumption that at most $f = (2\sigma - 1)n$ participants are faulty. Both protocols ensure safety when the number of faulty participants is at most $(2\sigma - 1)n = f$, and liveness when it is less than $(1 - \sigma)n = \frac{n-f}{2}$. For example, for $f = 0$, $\sigma = \frac{1}{2}$; for $f = \frac{1}{2}n$, $\sigma = \frac{3}{4}$; and if everyone but one agent is faulty then $\sigma = \frac{2n-1}{2n} = 1 - \frac{1}{2n}$, namely unanimity is required. For $n = 3f + 1$ (the standard assumption), $\sigma = \frac{n+\frac{n-1}{3}}{2n} = \frac{2}{3} - \frac{1}{6n}$, so a supermajority greater-or-equal to $\frac{2}{3}n$ is required. Typically, permissioned consensus protocols wait to hear from "at least $n - f$ agents among P", which in the standard case of $n = 3f + 1$ (but not otherwise) happens to be equivalent to "a σ-supermajority among P".

Constitution safety and liveness. The (amendable component of the) constitution thus defines the protocol's assumptions about the environment: A constitution (P, σ, Δ) is *safe* if P is the set of agents running the protocol; there is a correct σ-supermajority in P, and every σ-supermajority in P includes a majority of correct agents, hence the intersection of any two σ-supermajorities contains a correct agent. A constitution is *live* if fewer than $(1 - \sigma)n$ of its participants are faulty and after GST any message sent among correct participants arrives within Δ.

Constitution safety and liveness attest to the validity of the assumption of the constitution regarding the participants and the network connecting them. Note that Δ bounds the message delay among *correct* (i.e., participating and online) agents after GST, not the time until every agent comes online. The dual-mode design means the protocol is quiescent when there are no pending transactions, so agents need not be continuously available. When a transaction arrives, only a correct σ-supermajority must respond within Δ; agents that are temporarily offline are accounted for by the choice of σ.

Constitutional Consensus problem definition. A constitutional consensus protocol may have multiple *instances*. Each instance commences with an *initial constitution* (P, σ, Δ). As in standard consensus, participants spontaneously issue transactions (these are the inputs to

the protocol) and every agent outputs a sequence of transactions, called its *output sequence*.

In addition, Constitutional Consensus assumes that participants engage in decision processes as specified by the constitution, which may result in decisions to amend the prevailing constitution. As an example, Sections 4 and 5 show how known democratic decision processes can be applied in this new context. Note that the parameters P and σ are shared: they govern both the consensus protocol and the democratic decision processes themselves. The interface between the Constitutional Consensus protocol and the constitutional democratic decision processes is as follows: (*i*) a democratic process may produce constitutional amendment decisions, which become known to the participants once made; (*ii*) participants can verify that a constitutional amendment decision is *valid* according to the prevailing constitution, as specified in Section 5. A valid constitutional amendment decision defines a new constitution (P', σ', Δ'). An *initial constitution* is the new constitution of a valid amendment of a constitution with an empty population.

Constitutional amendment decisions are provided to the consensus protocol as inputs by participants that know of them. Thus, the protocol's output sequence includes both transactions and valid constitutional amendment decisions, as follows:

▶ **Definition 2** (Constitutional sequence, epoch, epoch sequence, prevailing constitution, participants)**.** *A* constitutional sequence *is a sequence $T = t_1, t_2, \ldots$ of transactions and valid constitutional amendment decisions beginning with an initial constitution. Each constitutional amendment decision d in the sequence defines a new* epoch*, its* epoch sequence *is a maximal contiguous subsequence of T that begins with d and does not include another such decision, and its* prevailing constitution *is the new constitution of d. An agent* participates *in an epoch if it is a member of the population of its prevailing constitution.*

The constitutional output-sequences produced by correct agents executing a constitutional consensus protocol instance must satisfy the following safety and liveness conditions:

▶ **Definition 3** (Constitutional Consensus Correctness)**.** *A constitutional consensus protocol is* correct *if, in any run of it in which all valid constitutional amendments are safe, there exists a constitutional sequence $T = t_1, t_2, \ldots$ for which the following properties hold:*

Consistency *Every correct agent's output sequence is a subsequence of T.*

Validity *If a correct agent p outputs $t \in T$, then p is a participant in t's epoch.*

Epoch Liveness *For a correct participant p in an epoch d in T, p's output eventually includes all the transactions in d's epoch sequence. Furthermore, if d is the last epoch in T, then every transaction input by a correct participant in epoch d is included in T.*

Constitution Amendment Liveness *If a valid constitutional amendment decision d is provided as input to the protocol by a correct agent, then T eventually includes d.*

Note that with no amendments, this reduces to the standard consensus correctness definition. Progress is guaranteed after GST even under infinitely many amendments, though frequent amendments may starve normal transactions; in practice, amendments are expected to be rare.

3 Single Epoch Constitutional Consensus

Here we present the operation of the Constitutional Consensus protocol during a single epoch. We overview the protocol's operation, deferring its formal description and correctness proofs to Appendices A and B, resp. We begin (Section 3.1) by explaining the finalisation process, which ensures the protocol's safety. We then discuss the conditions for generating

new protocol blocks so as to guarantee liveness (Section 3.2). Finally, Section 3.3 summarises the protocol's state, initialisation, and output.

3.1 Safety: Finalising Transactions

Blocklace-based ordering. The protocol is based on a *blocklace* [40], a cryptographic DAG data structure consisting of *blocks*, each of which includes a *payload* and hash-based *pointers* to other (earlier) blocks. A block is signed by its creator; a block created by agent p is called a *p-block*.

An *initial* block has no pointers. The DAG structure defines a partial order on blocks: we say that a block b *observes* a block b' in a blocklace B if there is a path from b to b' in B. (In particular, every block observes itself). The *depth* of a block b, $depth(b)$, is 0 if b is initial else $depth(b) = depth(b') + 1$ where b' is a maximal-depth block pointed to by b. The *closure* of a block b, denoted $[b]$, is the set of blocks observed by b.

The protocol employs an evolving shared blocklace, of which each participant $p \in P$ maintains a local copy B. In each epoch, the protocol creates a blocklace that begins with a *constitutional genesis block* b, whose payload specifies a new constitution $C = (P, \sigma, \Delta)$. Every participant commences the epoch with its copy of B holding $\{b\}$. During the epoch, participants create new blocks that observe their latest blocklace B and disseminate them to other participants. Blocks' payloads may include transactions the agents wish to order and also serve as protocol messages to drive agreement.

While a blocklace is only partially ordered, the protocol totally orders the blocks in an epoch (and the transactions included therein) in a consistent way, producing the same output sequence at all participants in an epoch. Moreover, epochs are also totally ordered, so that the totality of all blocks of all epochs are also totally ordered, even though not always by the same participants.

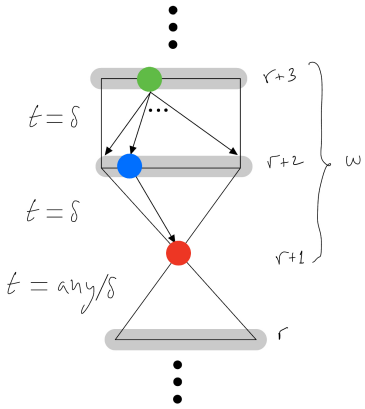
Waves, modes, and leaders. The blocklace is partitioned into a sequence of waves. For $k > 0$, the kth wave in B is the set of blocks of depths $3k - 2$, $3k - 1$, and $3k$ in B, referred to as the the *first*, *second*, and *third round* of the wave, respectively. The 0th wave of a blocklace is defined to be the singleton holding its constitutional genesis block.

The protocol has two operation modes, *high* and *low throughput*. In high throughput mode, agents continuously generate blocks in all rounds. To break the symmetry among agents, we use a deterministic (and fair) *leader* function that defines a unique *formal leader* $p_\ell \in P$ in the first round of each wave. Within each wave, the protocol strives to finalise a first-round p_ℓ-block (if it exists).

In low throughput mode, agents are *quiescent* (defined below) as long as they have no new transactions to send. The protocol only quiesces at the end of a wave, after all previously sent transactions have been ordered. The quiescent state can last for seconds, minutes, or even hours, as in small social communities, transactions might be infrequent. Once any participant has a transaction to order, it initiates the next wave by sending a first round block. All other agents join this wave by issuing second, and later third, round blocks. In this scenario, the single first round block in the wave is considered a *spontaneous leader*, and the protocol strives to finalise it.

Finalising blocks in waves. Finalisation within a wave proceeds in three steps. The first round of the wave includes candidates for finality, the second includes endorsements of candidates, and the third includes ratifications, as we now explain.

But first, we need to disregard *equivocations*, namely, two blocks by the same agent that do not observe each other. We say that a block b *approves* a block b' if b observes b' and



Round $r+1$ has a single leader block (red) approving a supermajority in round r (the last round of the previous wave). Round $r+2$ consists of a supermajority of blocks (blue as an example) pointing only to (and thus endorsing) the first-round leader block. Round $r+3$ consists of a supermajority of blocks (green as an example) each pointing to all the second-round blocks, and thus ratifying the first-round leader block, which is thus finalised by the wave.

Low-throughput good case: The wave ending in round r is quiescent, the spontaneous non-empty leader block (red) may be produced at any time after round r has advanced.

High-throughput good case: The formal leader produces a block (red) within δ after the previous round r is advanced.

Figure 1 A good-case wave w.

does not observe any block equivocating with b'. A second-round block b *endorses* at most one first-round block in its wave: if the preceding wave is quiescent, b endorses the sole first-round block it approves (if unique); if non-quiescent, b endorses the formal leader's block (if approved). A first-round block is *ratified* by a third-round block that approves a σ-supermajority of endorsing second-round blocks, and *final* once a σ-supermajority of third-round blocks ratify it. Since correct agents do not equivocate, at most one block per wave can be ratified or final. A finalising wave is illustrated in Figure 1.

We are now ready to define quiescence; the definition is mutually recursive with the definitions of endorsement, ratification, and finality. For the base, the 0th-wave is *quiescent* and its constitutional genesis block *final*. A subsequent wave is *quiescent* if it includes a final leader block b, and all the blocks in the wave except b have empty payloads. Thus, the protocol begins quiescent and remains in low throughput mode as long as there is at most one agent issuing transactions in each wave. It switches to high throughput mode whenever multiple agents send transactions in the same wave.

Ordering. As in other DAG-based protocols [39, 40, 50], once a leader block b is finalised, the protocol deterministically orders all the transactions approved by b that have not yet been totally ordered, and outputs the resulting sequence. Thus, as long as new blocks are being finalised, all the transactions in the system become ordered.

Ordering must be consistent at all agents. To this end, we leverage the property that at most one block is ratified or final in a wave. We define a recursive ordering function τ. When called for a block b, τ locates the most recent ratified and yet unordered block in $[b] \setminus \{b\}$ and recursively calls τ for this block. Once no such block exists, τ topologically sorts all unordered blocks approved by b.

3.2 Liveness: Dissemination and Round Progress Rules

In the previous section, we explained how transactions are finalised in the good case, when a first-round leader block is approved by a super-majority of endorsements in the second round and a super-majority of ratifications in the third.

We now discuss how we continue to progress through rounds when the good case does not occur, while taking care not to rush to the next round before giving the leader a chance to be finalised in case we are after GST.

To formalize these progress rules, we first define a round r as *advanced* in a blocklace B

if B includes enough blocks to allow round $r+1$ blocks to be added to the blocklace safely.

In second and third rounds, all correct participants send blocks, and these rounds advance once they include a super-majority of blocks. In a first round, only a leader (formal or spontaneous) sends a block in the good case. Hence a round is advanced once it includes a leader block. Going beyond the good case, if during high-throughput agents fail to receive a first-round leader block in a timely manner, they issue first-round blocks of their own. Once the blocklace includes a super-majority of round-one blocks, the round is advanced despite not including a leader block.

A second challenge is partial dissemination, whereby a faulty agent sends a block to some participants and not others, causing missing dependencies and potentially preventing some correct agents—including the formal leader—from advancing to the next round.

Partial dissemination is handled using two special *dissemination-inducing* blocks, which trigger block forwarding and are not added to the blocklace: a NACK-block for missing dependencies and an INFORM-block for an unresponsive leader.

Before sending dissemination-inducing blocks, we wait sufficiently long for messages from correct participants to arrive after GST, so as not to induce extra traffic in the good case. Similarly, we have to carefully set the timeout for sending non-leader first-round blocks so as to give a correct leader sufficient time to make up any missing blocks and send its new block. The delays used in the protocol are explained below.

3.3 Single-Epoch Constitutional Consensus Pseudocode

In the Constitutional Consensus protocol, each agent p maintains two sets of blocks: An input buffer D and a local blocklace B. An epoch identifier e points to the epoch's constitutional genesis block. B^e denotes the *blocklace of epoch e in B*. Algorithm 1 presents the single-epoch Constitutional Consensus pseudocode, in which the prevailing constitution is fixed throughout the run. We use the terminology *send/issue once* as a shorthand for send/issue if the block has not been sent/issued yet.

Pseudocode walkthrough. Receive: valid incoming ordinary blocks are inserted to D. Subsequently, the **Accept or Nack** is applied: if any of their predecessors are missing, p waits Δ to issue a NACK, and once a block in D has no missing predecessors it is *accepted* to B and removed from D.

Issue: The variable r always holds the highest advanced round in B, and an agent typically issues blocks in round $r+1$. This means that rounds can be skipped if the agent has been slow sending while the blocklace has advanced. This is useful when an agent that has been offline for some time catches up with the current blocklace before beginning to send blocks of its own. The only exception (to sending in round $r+1$) is the **Issue Backlog** rule: In case p has payload to send, has not sent a round r block, and is not supposed to send a round $r+1$ block, p issues a round r block to ensure its payload is disseminated.

If r is a first or second round in its wave, then p immediately (upon r advancing) issues a round $r+1$ block. If r is a third round, then block issuing (in the ensuing wave) depends on the mode. During low throughput (i.e., following a quiescent wave), p issues a block if and only if it has payload to send. During high-throughput, p issues a block in case (i) it is the formal leader, or (ii) it times-out on the leader. Here, a long timeout is used (9Δ), before which we inform the leader that the round has advanced through the **Inform Leader** rule: Upon timing out – with a short timeout of 2Δ – on the formal leader in the first round of a high-throughput wave, p sends an inform block to the formal leader in order to cause it to advance in case it is correct.

Algorithm 1 Constitutional Consensus for a single epoch e with prevailing constitution (P, σ, Δ). Code for agent $p \in P$.

Let $B := B^e$ herein and r be the maximal advanced round of the current B^e.

Input: Upon input a transaction T, $payload \leftarrow payload \cup \{T\}$

Receive: Upon receipt of a q-block b, $q \in P$
if b is a NACK-block for a p-block then judiciously send $[b]$ to q,
else if b is an INFORM-block and $[b] \not\subseteq B \cup \{b\}$
then send once a NACK-block for b to q,
else if b is a valid ordinary block then $D \leftarrow D \cup \{b\}$.

Accept or Nack: For every q-block $b \in D$, $q \in P$,
if $[b] \subseteq B \cup \{b\}$ then $B \leftarrow B \cup \{b\}$, $D \leftarrow D \setminus \{b\}$,
else if b was added to D more than Δ time ago
then send once a NACK-block for b to q.

Issue: Issue once a round $r+1$ block if $r+1$ is a second or third round or a first round that follows a wave that is:

1. quiescent and $payload \neq \perp$, or
2. non-quiescent and (i) $p = leader_P(r+1)$ or (ii) r has been advanced for 9Δ.

Issue Backlog: If $payload \neq \perp$ and no round r or round $r+1$ block has been issued by p, then issue once a round r block.

Inform Leader: If r is a third-round of a non-quiescent wave that has been advanced for 2Δ then send once an INFORM-block for round $r+1$ to $leader_P(r+1)$.

Output: If B contains a final block b of depth greater than the previous final block then output all the transactions in every block in $\tau(b)$ (Definition 12) that have not already been output.

- *issue a round k block* means create a block b with *payload* and pointers to the tips of B_{k-1}, add b to B, send b to every $q \neq p \in P$, and set $payload \leftarrow \perp$.
- *judiciously send $[b]$ to q* means send every $b' \in [b]$ to q unless (i) p has already sent b' to q, or (ii) $b' \in [b_q]$ for some q-block $b_q \in B \cup D$.

Delays. The protocol makes judicious use of delays, to ensure both liveness and efficiency. Here is the rationale for the various protocol delays, all expressed as multiples of Δ:

Δ for issuing nack-blocks. In the good case we wish no NACK-blocks to be sent at all. Agents wait Δ before complaining, so if a correct agent sends a block b to p and q at the same time, q may receive b up to Δ sooner than q and send a block b' that depends on b to p; but p will receive b within Δ from receiving b'. This delay is an optimization to avoid sending NACK-blocks in the good case; eliminating it does not hamper safety or liveness.

2Δ for informing the ensuing round's leader. In the good case whence all agents are correct after GST, whenever an agent receives a block, all other agents receive it within Δ time. Thus, once a third round is advanced at the blocklace of a correct non-leader agent p, p expects to receive a leader block of the ensuing (first) round within 2Δ. If this does not happen, this could be because (i) it is before GST, (ii) the leader is incorrect, or (iii) the leader has not seen yet that the previous round is advanced. This last third case can happen after GST even if the leader is correct, due to Byzantine partial dissemination of third-round blocks. Since p knows that the previous third round is advanced, it sends an

INFORM-block to the leader, which will cause the leader to send a NACK-block requesting the blocks it is missing. This 2Δ delay is an optimization to avoid spurious informs in the good case; eliminating it does not hamper safety or liveness.

9Δ for an unresponsive leader. Following a non-quiescent wave after GST, if the leader p_ℓ is correct, all correct agents should wait for p_ℓ's first round block before proceeding to the second round in order to allow that block to be finalized. But because p_ℓ may be faulty, they need to eventually time-out and stop waiting. To determine the time-out, we consider the worst-case latencies as follows: Assume a third round r is advanced at p at time t. If p does not receive a leader block by $t + 2\Delta$, it informs p_ℓ, which issues a NACK-block, upon receipt of which p sends p_ℓ any missing blocks. This exchange takes at most 3Δ, so by time $t + 5\Delta$, p_ℓ issues a leader block b for round $r + 1$, which is received by p by time $t + 6\Delta$. Upon receipt of b, p might be missing some blocks in $[b]$ and wait Δ before sending a NACK and getting them from q by time $t + 9\Delta$. If p does not accept any leader-block by $t + 9\Delta$, it concludes that either p_ℓ is faulty or GST has not arrived yet, and sends a first-round block of its own in order to allow the protocol to proceed to the next round.

Initialisation and input. The first epoch is initialised with $D \leftarrow \phi$, $B \leftarrow \{b\}$, $e \leftarrow b$, where b is a constitutional genesis block and $payload \leftarrow \perp$. As input transactions arrive, they are added to *payload*. **Output:** Once the protocol finalises a block b, it orders the unordered blocks approved by b using a recursive function τ defined in Appendix A.3, and outputs their transactions.

Latency. After GST and in the good case a leader block (formal or spontaneous) is finalised by its wave within 3δ, during both low- and high-throughput (See Figure 1).

Communication complexity. During low throughput, in the good case the spontaneous leader block is sent to all, constant-size endorsement blocks are sent all-to-all, and the culprit is the third round, where blocks each with $O(n)$-pointers are sent to all-to-all. In Morpheus [50], this issue is alleviated by having second-round blocks be replaced by threshold signatures on the first block (in low-throughput mode): a σ-supermajority of these signatures can be amalgamated to form a single signature of constant length, and then third-round blocks can be replaced with threshold signatures on that single signature, giving communication complexity $O(n^2)$. A similar idea can be applied here, by having hitherto-empty blocks carry a threshold signature instead. Specifically, a second-round block, instead of being empty, should carry as payload a threshold signature of the pointer to the first-round leader block. A third-round block that observes a supermajority of second-round blocks with threshold signatures, instead of being empty and pointing to all second-round blocks, would carry as payload an amalgamation of these threshold signatures, and point only to its own second-round block. Thus, in the good case, all blocks of a low-throughput wave are of constant size, resulting in $O(n^2)$ communication complexity. To accommodate this change, the definition of a third-round p-block b *ratifying* a first-round leader block b' has to include the case that b points only to the second round p-block, but its payload is an amalgamation of threshold signatures of a supermajority of second-round blocks that endorse b'.

During high-throughput, all blocks carry payloads. If the payloads have at least $O(n)$ transactions each, then, in the good case, the amortized communication complexity per transaction is $O(n)$.

4 Sybil-Resilient Democratic Decision Processes

The question of how a democratic constitution can be amended, democratically, is deep and fundamental [14].

Here we recall democratic decision rules for amending each component of the constitution (P, σ, Δ), grounded in computational social choice [11] and summarised by three properties:

1. *Reality-aware* [71]: The only type of decision is to amend the status quo; in particular, to amend the prevailing constitution [1]. Thus, the prevailing constitution (P, σ, Δ) is an ever-present reality: it governs the running consensus protocol regardless of whether anyone wishes to change it, with amendment rules treating it as the default and moving away from it only when there is sufficient support.
2. *Sybil-resilient* [65, **?**]: Change of the status-quo can be effected only by support large enough to necessarily reflect the will of the non-Sybil agents.[2]
3. *Aggregation over metric spaces with single-peaked domains* [16]: We assume that: (1) each agent has an ideal point in each of the decision domains; (2) the domain has a metric, so that an agent prefers any point closer to their ideal point over any point further away from it; and (3) there exists an aggregation function that, given the ideal points of agents, produces a point that corresponds to the agent opinions. This allows *unifying proposing and voting*: Agents just state their ideal points and the aggregation function identifies a most supported point; if those that prefer this point over the status-quo are a supermajority, it becomes the new decision.

These lines, together with deliberative coalition formation and AI mediation, are integrated into a general framework, *constitutional governance in metric spaces* [72], in which every component of a community's legal corpus—its laws as well as its constitution—is a point in a metric space amended by supermajority. Here we need only the concrete rules for the three constitutional parameters, as follows.

The amendment setting.

Following Bulteau et al. [16], there is no proposal phase. Whenever an agent decides to vote (afresh or as update), it issues a block with its timestamped and signed vote. A vote is *persistent*: if agent i does not update its vote, the most recent one in the blocklace remains in effect. The prevailing constitution thus always reflects the current preferences of all agents without requiring repeated re-voting. As we show next, the decision spaces of P, σ, and Δ are all naturally embedded in corresponding metric spaces.

Domain structure of the parameters. The three constitutional parameters have distinct domain structures, which determine the appropriate aggregation rule for each:

- P **(population):** For each potential participant p, each agent votes yes or no—effectively, a separate referendum on each member. This is the simplest form of status-quo voting.
- σ **(supermajority threshold):** Agents state a preferred value in $[0.5, 1)$, a naturally single-peaked domain. Additionally, σ is *self-referential*: it is the threshold used to amend the constitution, including σ itself. The amendment rule must therefore be consistent with its own output.
- Δ **(timeout):** Agents state a preferred value in $\mathbb{R}^+$, again a single-peaked one-dimensional domain.

Unified proposing and voting means that participants need only agree on a set of votes, and then each agent can independently apply the amendment rules (described next) to determine the next constitution. The protocol for agreeing on the set of votes is described in Section 5 below.

[2] Sybil-resilient admission protocols [59, 66] during community formation are complementary to this work.

Amendment rules. We now specify the concrete amendment rules given the agreed upon set of votes and the prevailing (P, σ, Δ); the updated constitution (P', σ', Δ') takes effect when the consensus protocol switches to a new epoch, as described in Section 5.

(1) Amending P. Each agent $i \in P$ approves independently each potential addition to or removal from P. A candidate $p \notin P$, known to the voters through real-world acquaintance, is added to P' if and only if a σ-supermajority of the prevailing P approves, and similarly a member $p \in P$ is excluded from P' if and only if a σ-supermajority of the prevailing P approves. Additionally, every new member in $P' \setminus P$ must consent to joining, to prevent deadlock from non-responsive members [43].

This per-member σ-supermajority rule follows directly from the Sybil-resilient setting of Shahaf et al. [65] (for the setting of voting in a binary domain-in/out), who show that it is safe with respect to honest agents: a minority of Sybils cannot force membership changes.

(2) Amending σ. Each agent $i \in P$ states its preferred $\sigma_i \in [0.5, 1)$. The value of σ is updated according to the *h-rule for constitutional amendment* [1] (named after the h-index of citation metrics), as follows: (1) *Raising σ:* σ is increased to the maximal $\sigma' > \sigma$ for which a σ'-supermajority of P voted for a value $\geq \sigma'$. (2) *Lowering σ:* σ is decreased to the minimal σ' for which a σ-supermajority of P voted for a value $\leq \sigma'$. (3) If neither condition holds, σ remains unchanged.

For example, changing σ from $\frac{2}{3}$ to $\frac{3}{4}$ requires the approval of a $\frac{3}{4}$-supermajority. Changing to unanimity requires approval by all members of P.

The h-rule is self-referential by design: raising the threshold to σ' requires a σ'-supermajority supporting it (preventing a minority from imposing stricter requirements), while lowering requires the *current* σ-supermajority to agree (preventing a minority from weakening the constitution). Abramowitz et al. [1] show that the h-rule is the unique rule satisfying a natural set of axioms for this self-referential setting.

(3) Amending Δ. Each agent $i \in P$ states its preferred $\Delta_i \in \mathbb{R}^+$. Since Δ is a one-dimensional, single-peaked domain with potential Sybil presence, we use the *Suppress Outer-f* parameter update rule of Shahaf et al. [65]:[3]

- If the median vote $> \Delta$: discard the maximal f votes, compute the median Δ' of the remaining votes, and update to Δ' if $\Delta' > \Delta$.
- Symmetrically, if the median vote $< \Delta$: discard the minimal f votes, compute the median Δ' of the remaining votes, and update to Δ' if $\Delta' < \Delta$.
- Otherwise, Δ remains unchanged.

The key properties: if all Sybil agents wish to increase Δ (to slow down the protocol) but the correct agents do not, Δ will not increase; symmetrically for decreasing. If all correct agents unanimously wish to update Δ to some Δ', the update succeeds regardless of adversarial votes, provided $f < \frac{n}{3}$.

[3] Equivalently, we can use the Reality-Enforcing rule of Meir et al. [**?**], which is defined differently but is mathematically equivalent. In the original reference [65] the rule refers to 'outer-σ'; we use 'outer-f' here to avoid confusion with our use of σ.

A constitution (P, σ, Δ) is amended democratically to (P', σ', Δ'), using the prevailing (P, σ, Δ) as status quo:

1. $P \to P'$: per-member σ-**supermajority** among P, plus **consent of all new members** in $P' \setminus P$ [65].
2. $\sigma \to \sigma'$: the h-**rule** for constitutional amendment [1].
3. $\Delta \to \Delta'$: the **Suppress Outer-f** parameter update rule [65].

5 Constitutional Consensus with Constitutional Amendments

We now extend the consensus protocol with constitutional amendments, which generalise reconfiguration in both scope (any protocol parameter, not just membership) and authority (decided by participants through constitutionally-specified processes, not by a leader or external entity); a comparison with reconfiguration work is given in Appendix G. The protocol's correctness is proved in Appendix E, and it being grassroots [68] in Appendix F.

In this work, the amendable component of the constitution consists of (P, σ, Δ) (Definition 1), parameters which affect the safety, liveness, and performance of almost any consensus protocol for eventual synchrony.

The democratic decision processes described in Section 4 are part of the constitution: they specify how each amendable component may be changed.

In each epoch of the consensus protocol, participants may use the *amendment protocol* to determine when and in what way to amend the constitution. Each participant *votes*, via the amendment protocol, for its preferred values for P, σ, and Δ. Participants vote both initially (upon invoking the amendment protocol) and whenever they change their preferences. Votes are signed and sent in blocks of the single-epoch consensus protocol. If a participant $p \in P$ votes to add a member $p' \notin P$ to the constitution, it includes the signature of p' on the vote in its vote block. A vote *takes effect* once it is ordered by the consensus protocol, and remains *in effect* until another vote of the same participant takes effect or a new epoch of the consensus protocol begins.

A participant p invokes the amendment protocol in epoch i if either p's preference for the constitution parameters has changed since its vote in epoch $i-1$, or another participant has already voted in epoch i, prompting p to join the amendment protocol. In order to avoid frequent fluctuations of the constitution, it is possible to rate-limit amendments by refraining from invoking the amendment protocol for some time after the epoch starts (for example, after a day or even a week). While Byzantine agents might send votes before the deadline, a supermajority of votes will not be received before correct agents invoke the amendment protocol, so, as we shall see below, no amendment will occur.

The amendment protocol tracks the votes as they take effect, and after each one, evaluates whether the constitution needs to be amended according to the set of votes ordered so far. There are three prerequisites for amending the constitution: (1) there is a σ-supermajority (in the prevailing constitution) of votes in effect in the current epoch; (2) based on these votes, the amendment rules of the democratic decision process (described above) determine that the constitution should change; and (3) this is the first time during the epoch when conditions (1) and (2) hold.

Once the amendment protocol determines the next constitution, it produces and signs a constitutional amendment decision:

▶ **Definition 4** (Constitutional Amendment Decision). *A* constitutional amendment decision *is a pair* $d = (h, x)$, *where* $x = (\textsc{amend}, Id, i, C, C')$, *Id is an* instance identifier, $i \geq 1$

is the epoch *within this instance,* C, C' *are constitutions, referred to as the* old *and* new *constitutions of* d*, resp., and* h *is a set of signed votes. If* $i = 1$ *then* $C = \bot$*.*

The decision d *is* valid *in epoch* i *of instance* Id *with prevailing constitution* $C = (P, \sigma, \Delta)$ *if (1)* h *includes a* σ*-supermajority in* P *of signed votes for epoch* i *in instance* Id*, (2) these are the first* σ*-supermajority of signed votes ordered in epoch* i *of the constitutional consensus protocol instance* Id *that lead to a constitution amendment decision, and (3)* C' *is the result of applying the amendment protocol to* h *with prevailing constitution* C*.*

Thanks to the consensus on votes and to the deterministic nature of the democratic decision process, the amendment protocol produces a unique valid amendment per epoch.

Constitutional amendments cause epoch transitions, which proceed in three phases. **Amendment finalisation:** Once the amendment protocol at a participant p results in a valid amendment d, p includes d as payload in every block it issues until d is finalised. **End epoch:** upon finalising a constitutional block with d, an agent produces a CORONATE block, sends it to both old and new populations, and ceases participation; it must still respond to belated NACK requests from agents finalising the old epoch. **Start epoch:** a member of the new population commences the new epoch upon receiving a σ-supermajority of CORONATE blocks (and having sent one, if also in the old population). The full pseudocode is given in Algorithm 2. Liveness of constitutional amendments is guaranteed after GST since all votes sent after GST become ordered, and every correct agent joins the amendment protocol once it is invoked.

Algorithm 2 Incorporating Constitutional Amendments in Constitutional Consensus. Code for any participant p in the prevailing constitution and for any $q \in \Pi$.

Amendment Finalisation by p: Upon issuing an ordinary block, if p's amendment protocol has produced a valid constitutional amendment decision d that is not final in B then issue the block with d instead of with *payload*.

End Epoch by p: Upon finalising in B a constitutional block b with constitutional amendment d, prevailing population P, and new population P', call $\tau(b)$, create a $(\text{CORONATE}, d)$ block, send it to $P \cup P'$, and cease participation in the epoch.

Start Epoch by q: Let d be a valid decision to amend the prevailing constitution, with new population P'.
Upon receipt by $q \in P'$ of a σ-supermajority among P of $(\text{CORONATE}, d)$ blocks, and provided q previously issued such a block if $q \in P$, commence participation in the new epoch by $e \leftarrow d$ as the new epoch constitutional genesis block, $B \leftarrow B \cup \{d\}$, and retaining D and *payload*.

Respond to Belated nack by q: Upon receipt of a NACK q'-block for a q-block b of a past epoch e for which $q' \in \Pi$ is a participant, judiciously send $[b]$ to q'.

In the good case, a new constitutional amendment d is issued by the leader of a wave and finalised within that same wave. In the worst case, it might take arbitrarily many waves to finalise d prior to GST or due to faulty leaders. After GST, a constitutional amendment block issued by a correct leader among P is bound to be finalised (and such a leader is bound to be selected eventually due to the fairness of $leader_P$). Once finalised, correct members of P will issue a CORONATE block and correct members of P' will receive it, allowing the current epoch to end and the next one to begin. This argument informally establishes the liveness of the transition among constitutions due to constitutional amendments.

6 Related Work

A full survey of related work is provided in Appendix G, organised into five areas. It reviews smart contracts and digital social contracts [26, 33, 20]; DAOs and blockchain governance, including self-amending blockchains [35, 79, 23, 42, 19], DAO frameworks [24, 48, 57, 54, 58], critiques of plutocratic governance [63, 73, 75, 17], and consortium platforms [3, 13]; reconfiguration of consensus protocols [43, 46, 45, 2, 74, 53, 56, 9, 28, 41, 77, 8]; BFT consensus protocols, both leader-based [21, 80, 15] and DAG-based [39, 25, 32, 31, 4, 60], including the two protocols on which Constitutional Consensus builds [40, 50]; and grassroots protocols and democratic identity, including centralised democratic platforms [5, 52, 62, 22, 6] and proof-of-personhood systems [30, 10, 38, 36, 12]. Constitutional Consensus combines three properties—democratic one-person-one-vote governance, decentralised BFT consensus, and self-governing constitutional amendment—and to the best of our knowledge, no existing system achieves all three.

7 Conclusions

The Constitutional Consensus protocol was devised to serve the needs of future grassroots platforms that operate digital social contracts. A key design principle is that governance is democratic: participants themselves decide on constitutional amendments—who participates, the supermajority threshold, and the timeout—through decision processes grounded in computational social choice [72]. Initially, we expect low-throughput applications to prevail, as small digital communities engaged in constitutional democratic conduct use the platform to grow and evolve. As communities federate to form ever-larger communities, and local community banks federate to form regional and national banks, higher-throughput execution would follow. Until then, simulating the various scenarios would be productive future work. Another direction is making the democratic decision-making process an amendable component of the constitution, making the system reflexive: the amendment process would then be subject to amendment through the very process it specifies.

AI Disclosure. We used Anthropic's Claude to assist with LaTeX editing, citation verification, manuscript restructuring, and programming as described in the paper. The authors verified the correctness and originality of all content including references.

References

1 Ben Abramowitz, Ehud Shapiro, and Nimrod Talmon. In the beginning there were n agents: Founding and amending a constitution. In *Algorithmic Decision Theory (ADT 2021)*, volume 13023 of *Lecture Notes in Computer Science*, pages 119–131. Springer, 2021.

2 Marcos K Aguilera, Idit Keidar, Dahlia Malkhi, and Alexander Shraer. Dynamic atomic storage without consensus. *Journal of the ACM (JACM)*, 58(2):1–32, 2011.

3 Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. Hyperledger Fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference (EuroSys)*, pages 30:1–30:15, 2018.

4 Kushal Babel, Andrey Chursin, George Danezis, Lefteris Kokoris-Kogias, and Alberto Sonnino. Mysticeti: Low-latency dag consensus with fast commit path. *arXiv preprint arXiv:2310.14821*, 2023.

5 Xabier E. Barandiaran, Antonio Calleja-López, Arnau Monterde, and Carol Romero. *Decidim, a Technopolitical Network for Participatory Democracy: Philosophy, Practice and Autonomy of a Collective Platform in the Age of Digital Intelligence*. SpringerBriefs in Political Science. Springer, 2024.

6 Jan Behrens, Axel Kistner, Andreas Nitsche, and Björn Swierczek. The LiquidFeedback blockchain. *The Liquid Democracy Journal*, (6):18–29, 2018.

7 Juan Benet. Ipfs-content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*, 2014.

8 Alysson Bessani, Eduardo Adílio Pelinson Alchieri, João Sousa, André Oliveira, and Fernando Pedone. From Byzantine replication to blockchain: Consensus is only the beginning. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 424–436, 2020.

9 Alysson Bessani, João Sousa, and Eduardo E. P. Alchieri. State machine replication for the masses with BFT-SMART. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 355–362. IEEE, 2014.

10 Maria Borge, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, and Bryan Ford. Proof-of-personhood: Redemocratizing permissionless cryptocurrencies. In *2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 23–26. IEEE, 2017.

11 Felix Brandt, Vincent Conitzer, Ulle Endriss, Jérôme Lang, and Ariel D Procaccia. *Handbook of computational social choice*. Cambridge University Press, 2016.

12 Alain Brenzikofer. Encointer – local community cryptocurrencies with universal basic income. *CoRR*, abs/1912.12141, 2019.

13 Richard Gendal Brown, James Carlyle, Ian Grigg, and Mike Hearn. Corda: An introduction. Technical report, R3, 2016.

14 James M. Buchanan and Gordon Tullock. *The Calculus of Consent: Logical Foundations of Constitutional Democracy*. University of Michigan Press, 1962.

15 Ethan Buchman. Tendermint: Byzantine fault tolerance in the age of blockchains. Master of applied science thesis, University of Guelph, 2016.

16 Laurent Bulteau, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Aggregation over metric spaces: Proposing and voting in elections, budgeting, and legislation. *Journal of Artificial Intelligence Research*, 70:1413–1439, 2021.

17 Vitalik Buterin. Moving beyond coin voting governance. Blog post, August 2021.

18 Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang, and Yan X Zhang. Combining ghost and casper. *arXiv preprint arXiv:2003.03052*, 2020.

19 Jan Camenisch, Andrea Cerulli, David Derler, Manu Drijvers, Maria Dubovitskaya, Jens Groth, Timo Hanke, Gregory Neven, Yvonne-Anne Pignolet, Victor Shoup, Björn Tackmann, and Dominic Williams. The internet computer for geeks. Cryptology ePrint Archive, Paper 2022/087, 2022.

20 Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, volume 2781, pages 51–60. CEUR-WS, 2020.

21 Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*, pages 173–186. USENIX Association, 1999.

22 Consul Democracy Foundation. CONSUL DEMOCRACY — the most complete citizen participation tool for an open, transparent and democratic government. `https://consuldemocracy.org/`, 2025.

23 Jared Corduan, Andre Knispel, Matthias Benkort, Kevin Hammond, Charles Hoskinson, and Samuel Leathers. CIP-1694: A first step towards on-chain decentralized governance. Cardano Improvement Proposal, 2023.

24 Luis Cuende and Jorge Izquierdo. Aragon network: A decentralized infrastructure for value exchange. Whitepaper, 2017.

25 George Danezis, Eleftherios Kokoris Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. *arXiv preprint arXiv:2105.11827*, 2021.

26 Primavera De Filippi, Chris Wray, and Giovanni Sileno. Smart contracts. *Internet Policy Review*, 10(2), 2021.

27 Qinxu Ding, Daniel Liebau, Zhiguo Wang, and Weibiao Xu. A survey on decentralized autonomous organizations (daos) and their governance. *World Scientific Annual Review of Fintech*, 1:2350001, 2023.

28 Sisi Duan and Haibin Zhang. Foundations of dynamic BFT. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1317–1334. IEEE, 2022.

29 Ethereum. Decentralized autonomous organizations (DAOs) | ethereum.org, 2021.

30 Bryan Ford. Identity and personhood in digital democracy: Evaluating inclusion, equality, security, and privacy in pseudonym parties and other proofs of personhood. *CoRR*, abs/2011.02412, 2020.

31 Rati Gelashvili, Lefteris Kokoris-Kogias, Alberto Sonnino, Alexander Spiegelman, and Zhuolun Xiang. Jolteon and ditto: Network-adaptive efficient consensus with asynchronous fallback. In *Financial Cryptography and Data Security – 26th International Conference, FC 2022*, volume 13411 of *Lecture Notes in Computer Science*, pages 296–315. Springer, 2022.

32 Neil Giridharan, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Bullshark: Dag bft protocols made practical. *arXiv preprint arXiv:2201.05677*, 2022.

33 LM Goodman. Tezos: A self-amending crypto-ledger position paper. *Aug*, 3:2014, 2014.

34 Daniel Halpern, Ariel D Procaccia, Ehud Shapiro, and Nimrod Talmon. Federated assemblies. *Proc AAAI 2025; arXiv preprint arXiv:2405.19129*, 2024.

35 Samer Hassan and Primavera De Filippi. Decentralized autonomous organization. *Internet Policy Review*, 10(2):1–10, 2021.

36 Idena Team. Idena: Proof-of-person blockchain. Whitepaper, 2018.

37 Johannes Rude Jensen, Victor von Wachter, and Omri Ross. An introduction to decentralized finance (defi). *Complex Systems Informatics and Modeling Quarterly*, (26):46–54, 2021.

38 Dato Kavazi, Victor Smirnov, Sasha Shilina, Jonathan Shomroni, Rafael Contreras, Hardik Gajera, and Dmitry Lavrenov. Humanode: The first crypto-biometric network: Decentralized sybil resistance through biometric identity verification. In *Proceedings of the 2023 4th Asia Service Sciences and Software Engineering Conference (ASSE 2023)*. ACM, 2023.

39 Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. All you need is dag. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 165–175, 2021.

40 Idit Keidar, Oded Naor, and Ehud Shapiro. Cordial miners: A family of simple and efficient consensus protocols for every eventuality. In *37th International Symposium on Distributed Computing (DISC 2023)*. LIPICS, 2023. arXiv preprint arXiv:2205.09174.

41 Jovan Komatovic, Andrew Lewis-Pye, Joachim Neu, Tim Roughgarden, and Ertem Nusret Tas. From permissioned to proof-of-stake consensus. In *7th Conference on Advances in Financial Technologies (AFT 2025)*, volume 354 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:26. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025.

42 Jae Kwon and Ethan Buchman. Cosmos: A network of distributed ledgers. Whitepaper, 2016.

43 Leslie Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, May 1998.

44 Leslie Lamport. Paxos made simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)*, pages 51–58, 2001.

45 Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Vertical Paxos and primary-backup replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 312–313. ACM, 2009.

46 Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Reconfiguring a state machine. *SIGACT News*, 41(1):63–73, 2010.

47 Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.

48 Robert Leshner. Compound governance. Blog post, Compound Labs, 2020.

49 Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. Grassroots flash: A payment system for grassroots cryptocurrencies. *arXiv preprint arXiv:2309.13191*, 2023.

50 Andrew Lewis-Pye and Ehud Shapiro. Morpheus consensus: Excelling on trails and autobahns. *Proc. 29th Conference on Principles of Distributed Systems, OPODIS 2025. arXiv preprint arXiv:2502.08465*, 2025.

51 Andy Lewis-Pye and Ehud Shapiro. Volition-guarded multiagent atomic transactions: Describing people and their machines. *arXiv preprint arXiv:2604.25596*, 2026.

52 Loomio Cooperative Limited. Loomio — make decisions together. `https://www.loomio.com/`, 2024.

53 Jacob R. Lorch, Atul Adya, William J. Bolosky, Ronnie Chaiken, John R. Douceur, and Jon Howell. The SMART way to migrate replicated stateful services. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys '06)*, pages 103–115. ACM, 2006.

54 Fabien Marino and Snapshot Labs. Snapshot: Off-chain gasless multi-governance client. Open-source software, 2020.

55 Satoshi Nakamoto and A Bitcoin. A peer-to-peer electronic cash system, 2008.

56 Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319. USENIX Association, 2014.

57 OpenZeppelin. Governor: On-chain governance. OpenZeppelin Contracts Documentation, 2021.

58 Optimism Foundation. The future of Optimism governance. Blog post, Optimism Collective, 2022.

59 Ouri Poupko, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Building a sybil-resilient digital community utilizing trust-graph connectivity. *IEEE/ACM transactions on networking*, 29(5):2215–2227, 2021.

60 Mayank Raikwar, Nikita Polyanskii, and Sebastian Müller. SoK: DAG-based consensus protocols. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–18. IEEE, 2024.

61 Sean Rhea, Brighten Godfrey, Brad Karp, John Kubiatowicz, Sylvia Ratnasamy, Scott Shenker, Ion Stoica, and Harlan Yu. Opendht: a public dht service and its uses. In *Proceedings of the 2005 conference on Applications, technologies, architectures, and protocols for computer communications*, pages 73–84, 2005.

62 Sonia Royo, Vicente Pina, and Jaime Garcia-Rayado. Decide Madrid: A critical analysis of an award-winning e-participation initiative. *Sustainability*, 12(4):1674, 2020.

63 Nathan Schneider. Cryptoeconomics as a limitation on governance. Mirror, 2022. Working paper.

64 Patrick Schueffel. Defi: Decentralized finance-an introduction and overview. *Journal of Innovation Management*, 9(3):I–XI, 2021.

65 Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Sybil-resilient reality-aware social choice. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pages 572–579, 2019.

66 Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Genuine personal identifiers and mutual sureties for sybil-resilient community growth. In *International Conference on Social Informatics*, pages 320–332, EU, 2020. Springer, Springer.

67 Ehud Shapiro. Grassroots currencies: A foundation for a grassroots digital economy. *arXiv preprint arXiv:2202.05619*, 2022.

68 Ehud Shapiro. Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*, pages 47:1, 47:7, Italy, 2023. LIPICS.

69 Ehud Shapiro. Grassroots social networking: Serverless, permissionless protocols for twitter/linkedin/whatsapp. In *OASIS '23*. Association for Computing Machinery, 2023.

70 Ehud Shapiro. Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*, pages 71–81, 2026. arXiv preprint arXiv:2502.11299.

71 Ehud Shapiro and Nimrod Talmon. Reality-aware social choice. *arXiv preprint arXiv:1710.10117*, 2017.

72 Ehud Shapiro and Nimrod Talmon. Constitutional governance in metric spaces. *Proceedings of the International Conference on Algorithmic Decision Theory (ADT 2026), to appear*, 2026. arXiv:2605.13362.

73 Divya Siddarth, Sergey Ivliev, Santiago Siri, and Paula Berman. Who watches the watchmen? A review of subjective approaches for sybil-resistance in proof of personhood protocols. *Frontiers in Blockchain*, 3:590171, 2020.

74 Alexander Spiegelman, Idit Keidar, and Dahlia Malkhi. Dynamic reconfiguration: Abstraction and optimal asynchronous solution. In *31st International Symposium on Distributed Computing (DISC 2017)*, volume 91 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 40:1–40:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017.

75 Nimrod Talmon. Social choice around decentralized autonomous organizations: on the computational social choice of digital communities. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pages 1768–1773, 2023.

76 Nimrod Talmon and Ehud Shapiro. Grassroots federation: Fair democratic governance at scale. *arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26*, 2025.

77 Guillaume Vizier and Vincent Gramoli. ComChain: A blockchain with Byzantine fault-tolerant reconfiguration. *Concurrency and Computation: Practice and Experience*, 32(12):e5494, 2020.

78 Qin Wang, Rujia Li, Qi Wang, and Shiping Chen. Non-fungible token (nft): Overview, evaluation, opportunities and challenges. *arXiv preprint arXiv:2105.07447*, 2021.

79 Gavin Wood. Polkadot: Vision for a heterogeneous multi-chain framework. White paper, Web3 Foundation, 2016.

80 Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proc. ACM PODC'19*, pages 347–356, 2019.

A Formal Definitions

A.1 Blocklace Preliminaries

We use $a \neq b \in X$ as a shorthand for $a \in X \wedge b \in X \wedge a \neq b$.

A *p-block* b, with $p \in \Pi$, is a triple $b = (h, x, H)$ where h is the hash of the pair (x, H) signed by p, referred to as the *identifier* of b, x is an arbitrary *payload*, which may be *empty*, $x = \bot$, in which case we refer to b as an *empty block*, and H a finite set of block identifiers, the blocks of which are *pointed to* by b. A block with no pointers, $H = \emptyset$, is called *initial*, in which case it can also be written as a pair $b = (h, x)$. An encoding of a block via a sequence of bits in an agreed-upon form is referred to as a *well-formed block*. We assume the hash function to be collision-free whp and cryptographic, so that blocks cannot form cycles and we may identify a block b with its identifier h. Furthermore, the method of signature allows the public key p to be recovered from a signature by p.

A set of blocks X is referred to as a *σ-supermajority given a set of agents P* if there is a σ-supermajority $Q \subseteq P$ such that for every $q \in Q$, X includes at least one q-block.

A block b *observes* itself, any block b' pointed to by b, and any block observed by b'. The 'observes' relation induces a partial order on any set of blocks. Two blocks that do not observe each other are *conflicting*, and if by the same agent they are *equivocating*. A block b *approves* a block b' if b observes b' and does not observe any block equivocating with b', and a set of blocks B *approves* b' if every $b \in B$ approves b'.

The *closure* of a block b, denoted $[b]$, is the set of blocks observed by b. For a set of blocks B, its *closure* $[B]$ is defined by $[B] := \bigcup_{b \in B} [b]$, which is also the set of blocks *observed by* B. A set of blocks B is *closed* if $[b] \subseteq B$ for every $b \in B$, equivalently if $B = [B]$. A *blocklace* is a closed set of blocks.

To help avoid confusion, we use B to denote a closed set of blocks (a blocklace) and D or X to denote any set of blocks, not necessarily closed. Also, we refer to a set of blocks by agents in $P \subset \Pi$ as *P-blocks*, and we let *the P-blocks in B* denote the set of all blocks in B by agents in P.

Given a set of blocks D, a pointer to a block b is *dangling* in D if $b \notin D$. Thus, a blocklace is a set of blocks with no dangling pointers. A block $b \in D$ is a *tip* of D if no other block $b' \neq b \in D$ observes b.

The *depth* of a block b, $depth(b)$, is 0 if b is initial else $depth(b) = depth(b') + 1$ where b' is a maximal-depth block pointed to by b. The *depth* of a set of blocks B is defined by $depth(B) := \max_{b \in B} depth(b)$. In the context of blocks, we use *most-recent* as synonym for deepest. A *round* $r \geq 0$ in B is the set of all blocks $b \in B$ with $depth(b) = r$, and the *r-prefix of B*, B_r, consists of all blocks $b \in B$ with $depth(b) \leq r$, equivalently all rounds in B up to and including r. Note that blocks of the same round do not observe each other.

A.2 Protocol concepts

Overview. We begin by disregarding *equivocations*, namely, two blocks by the same agent that do not observe each other. We say that a block b *approves* a block b' if b observes b' and does not observe any block equivocating with b'. We use this notion to define endorsements and ratification.

A second-round block b of a wave may approve multiple first-round blocks in its wave, but endorses (towards finality) at most one of them, depending on the mode. The mode is determined according the quiescence property of the wave W that precedes b in $[b]$, which we shall define later. If W is quiescent and b approves exactly one round-one block in its wave,

b', then b *endorses* b'. If W is non-quiescent and b approves a formal leader's round-one block, b', in its wave, then b *endorses* b'. In case neither condition is satisfied, b does not endorse any block.

A first-round block b is *ratified* by a third-round block approving a σ-supermajority in P of second-round blocks that endorse b. And a first-round block b is *final* once there is a σ-supermajority in P of third-round blocks that ratify b.

Figure 1 illustrates a "good case" wave in which the leader block is finalised. Observe that a wave can have at most one ratified or final block. This is because a second-round block endorses at most one first-round block, and since correct agents do not equivocate, there cannot be two σ-supermajorities in the same round endorsing different blocks.

For a given $P \subset \Pi$, the function $leader_P$ maps integers to P. For each first round of a wave it assigns a *formal leader* $p \in P$, where a first-round p-block b is a *leader block* if $p = leader_P(depth(b))$. We assume the leader function is *fair*, in that for every wave and every $p \in P$ there is a subsequent wave for which p is the formal leader of its first round.

▶ **Definition 5** (Endorse). *Let b' be a second round block that approves a first round block b in a wave W in the blocklace $B = [b']$. Let (P, σ) be the prevailing constitution.*

Then b' endorses *b if the most-recent wave $W' \subset B$ that precedes b is:*

1. ***quiescent*** *and b is the only first-round block approved by b', or*
2. ***non-quiescent*** *and b is a $leader_P$ block approved by b'.*

A set of second-round blocks $X \subset W$ endorses *b if every $b' \in X$ endorses b.*

Note that endorsement is an invariant block property, as it depends only on $[b']$. Next, we specify the finality rule. A first-round block $b \in W$ is final if the third round of W has a σ-supermajority that approves a second-round σ-supermajority that endorses b. Formally:

▶ **Definition 6** (Ratified, Final, Ordered, Quiescent). *Assume a blocklace B and a wave $W \subseteq B$. A first-round block $b \in W$ with prevailing constitution (P, σ) is:*

1. ratified by *a third-round block $b' \in W$ if b' approves a σ-supermajority in P that endorses b, in which case b is also* ratified in *W and in B; and*
2. final *in W and in B if the third-round of W includes a σ-supermajority in P, referred to as a* finalising σ-supermajority*, in which each block ratifies b.*

The wave W is:

1. finalising *if it includes a block b that is final in W; and*
2. quiescent *if, in addition, all blocks in W except perhaps b are empty, and no block in B conflicts with b.*

In addition, the 0th-wave is quiescent *and its constitutional genesis block* final.

Note that since only first-round blocks are endorsed, they are the only ones that can be ratified or final. Notice our use of the term "finalised" only applies to such leader blocks; the finalisation of leader blocks induces an order on all the blocks they approve, which are then said to be *ordered*. We observe that the block properties defined above are *monotonic*:

▶ **Observation 7** (Monotonicity). *If a block is ratified or final in some blocklace B, it is also ratified or final, respectively, in any blocklace $B' \supset B$.*

Note, however, that quiescence is not monotonic, because conflicting and non-empty blocks may be added to a round. The next observation asserts that a wave's final block is unique.

To formalize the round progress rules, we define a round r as *advanced* in a blocklace B if B includes enough blocks to allow round $r + 1$ blocks to be added to the blocklace safely:

▶ **Definition 8** (Advanced Round). *Assume a constitutional blocklace B with prevailing constitution (P, σ). A round r in a wave $W \subseteq B$ is* advanced *if it is a:*

1. *first round that either (i) includes a σ-supermajority of blocks among P, or (ii) includes a formal leader block or (iii) is preceded by a quiescent wave in B and includes at least one block, or a*
2. *second or third round that includes blocks by a σ-supermajority among P.*

Round 0 is trivially advanced.

Agents incorporate blocks in the blocklace only if they are valid, namely could have been produced by a correct agent during the run of the protocol. Formally:

▶ **Definition 9** (Valid Block). *A block b is* valid *if it is well-formed and round depth$(b) - 1$ is advanced in $[b]$.*

Dissemination. The protocol uses two types of *dissemination-inducing blocks*: NACK and INFORM; other blocks are referred to as *ordinary*. Dissemination-inducing blocks are not incorporated in the blocklace of their sender or recipient. Hence, they contribute only to the liveness of the protocol and do not affect its safety, which is derived from structural properties of the blocklace. Unlike Cordial Miners, which incorporates a blocklace-based dissemination protocol with $O(n^2)$ amortized communication complexity in the good case, Constitutional Consensus achieves for this model $O(n)$ amortized communication complexity in the good case during high-throughput.

Protocols such as TCP achieve the reliable transfer of a sequence of bytes from the sender to the recipient by employing ACK and NACK messages for transmission control. The purpose of an ACK message is to acknowledge the receipt of a datagram, and of a NACK message to report an out-of-order datagram to its sender, e.g., in the case of receiving the $(k+1)^{th}$-datagram before receiving the kth-datagram.

In the context of the blocklace, an 'out-of-order' block is a block received before some block it points to, and the purpose of a NACK-block is indeed to report of such an out-of-order block to its creator (which is not necessarily its sender).

NACK blocks are used to achieve reliable broadcast of ordinary blocks. They are needed in order to rectify partial dissemination by Byzantine agents. For example, consider a Byzantine agent that sends a block b to correct agent p but not to another correct agent q, and p adds b into its blocklace and sends a block b' that depends on b. Then when q receives b', it cannot add it to its blocklace because its predecessor b is missing. To this end, q sends a NACK block to p specifying the blocks it is missing.

Note that out-of-order blocks may naturally arise on a reliable network even during periods of synchrony and even when there are no Byzantine agents, because there is no bound on how *quickly* messages can arrive — in the example above, b's sender might be correct and b may still be underway to q. We therefore introduce a delay before sending a NACK. The result is that in the good case, when the network is synchronous and there is no partial-dissemination by Byzantine agents, every correct agent sends its newly created blocks to all other agents and no NACK-blocks or other communication is needed.

The following definition relates to a setting in which an agent p has a blocklace B and it receives a new block b. If $[b] \not\subseteq B \cup \{b\}$, then p cannot incorporate b into B since some pointers in b are dangling in B. These are the tips of $[b] \setminus (B \cup \{b\})$. Hence the following definition:

▶ **Definition 10** (NACK-block). *Given a blocklace B, a* NACK-block *by p for a block $b = (h, x, H)$ such that $[b] \not\subseteq B \cup \{b\}$ is the p-block $b' = (h', x', H')$ where $x' = (\text{NACK}, h)$ and*

$H' \neq \emptyset$ contains the tips of $[b] \setminus (B \cup \{b\})$.

Note that as defined above, $[b'] \subseteq [b]$, and if b has no pointers dangling in B then a NACK-block for b and B is undefined. If the NACK-block b' is defined, then its pointers H' are precisely the non-empty subset of H that is dangling in B.

The reason for the asymmetry of having NACK-blocks but not ACK-blocks is that any ordinary p-block functions as an ACK-block, as it attests to all the blocks known to p at the time of its creation. NACK blocks enable pull-based forwarding, where an agent who knows it misses a block asks for it. In addition, we make use of push-based forwarding to facilitate leader progress using INFORM-blocks.

▶ **Definition 11** (INFORM-Block). *Given a blocklace B, an* INFORM-block (h, x, H) *for round $r + 1 > 1$ has payload $x =$ INFORM and pointers H to the blocks of depth r in B.*

If an agent p suspects that the leader of the next round $r + 1$ does not proceed since it is missing some round r blocks known to p, then p sends the leader an INFORM-block pointing to the r blocks in its blocklace. If the leader indeed misses any of these blocks, which may happen even after GST due to Byzantine partial dissemination, it will respond to p with a NACK-block, which will in turn cause p to send to the leader the blocks it needs.

A.3 The Ordering Function τ

▶ **Definition 12** (τ). *Assume a safe constitution (P, σ) and any fixed topological sort function $xsort(b, B, B')$ (exclude and sort) that takes a block b and two blocklaces $B \supset B'$, and returns a sequence that preserves the 'observes' partial order and includes all the non-empty blocks in $B \setminus B'$ that are approved by b. The function τ takes a block as input and returns a sequence of blocks in $[b]$ as output, as follows:*

$$\tau(b) := \begin{cases} \tau(b') \cdot xsort(b, [b], [b']) & \text{if } b' \text{ is the most-recent block ratified in } [b] \\ xsort(b, [b]) & \text{if no such } b' \text{ exists} \end{cases}$$

When τ is called with a block b, it makes a recursive call with a block b' ratified in $[b]$. Note that b' is not necessarily final in $[b]$, nor final in the closure of the final block with which τ was called initially. In the implementation of τ, the results of recursive calls can be cached and used in subsequent calls, saving the need to recompute them.

B Safety and Liveness of Single-Epoch Constitutional Consensus

Here we prove the safety and liveness of the single-epoch Constitutional Consensus protocol presented in Algorithm 1. In this case, the members of the founding constitution $C = (P, \sigma, \Delta)$ participate in the protocol for the entire run. The output sequence at every correct participant begins with C and does not include additional constitutions.

B.1 Safety

In this section, we prove the single-epoch protocol's safety.

▶ **Theorem 13.** *If the founding constitution is safe then single-epoch Constitutional Consensus is safe.*

The safety properties (Consistency and Validity) in Definition 3 can be shown by proving the consistency of the output sequences of correct agents, formally defined as follows: We say that two sequences x and y are *consistent* if one is a prefix of the other, namely there is a sequence z such that $x \cdot z = y$ or $x = y \cdot z$, with $\cdot$ denoting sequence concatenation.

▶ **Observation 14.** *If the output sequences of every two correct agents are consistent with each other, then single-epoch consensus ensures safety.*

Proof. Validity trivially holds because every agent participates in every epoch. Assuming the output sequences of correct agents are all consistent with each other, we take T to be their supremum and get Consistency. ◀

The next observation asserts that a wave's final block is unique.

▶ **Observation 15** (Uniqueness)**.** *A wave can have at most one ratified or final block.*

Proof. The observation follows from the fact that a second-round block endorses at most one first-round block in any blocklace it is included in, so to have two σ-supermajorities of the same round endorse two blocks, at least one correct agent must endorse two blocks in this round; to do so this correct agent must equivocate, a contradiction. ◀

To prove consistency, first, recall the assumption that in a blocklace B created by the protocol with prevailing constitution (P, σ), the P-blocks in B include equivocations by at most $f = (2\sigma - 1)n$ agents. We refer to blocklaces and waves satisfying this property as *constitutionally-safe* wrt (P, σ).

▶ **Proposition 16.** *If the prevailing constitution is safe then $\tau(b)$ does not include equivocations.*

Proof. Consider an equivocation $b_1, b_2 \in [b]$. Consider the first call $xsort(b', [b'], [b''])$ for which at least one of b_1 or b_2 is in $X := [b'] \setminus [b'']$, and the following two cases:

1. Both $b_1, b_2 \in X$: Then b' approves neither hence neither will be in the output of *xsort* and hence nor in the output of τ.
2. Only one of them is in X, wlog $b_1 \in X$, and since this is the first such call to *xsort* it must be that $b_2 \in [b'']$. Then b' observes both b_1 and b_2 and hence will approve neither, whereas b'' observes only b_2, therefore will approve it and therefore only b_2 will occur in the output of τ, if at all (assuming there are no other equivocations with b_2).

In either case, not both b_1 and b_2 will be in $\tau(b)$. ◀

For the following proofs we generalize τ to blocklaces, so that $\tau(B)$ is $\tau(b)$ if b is the most-recent final block in B if there is one, else it is the empty sequence Λ.

▶ **Definition 17** (τ Safety)**.** *A blocklace B is* τ-safe *if for every wave $W \subset B$ with a block b final in W and any subsequent wave $W' \subset B$ with a block b' ratified in W', b is ratified in $W \cap [b']$.*

▶ **Proposition 18.** *A constitutional blocklace with a safe constitution is τ-safe.*

Proof. Let B be a constitutional blocklace with a safe constitution (P, σ), $W \subset B$ a wave with a block b final in W, and $W' \subset B$ a subsequent wave with a block b' ratified in W'. We have to show that b is ratified in $W \cap [b']$.

Since b' is ratified by some block it means that it is valid, since correct agents do not include invalid blocks in their blocklace, let alone endorse them, and a σ-supermajority

includes (a majority of) correct agents. Hence, being a first-round block of a wave, b' must observe a σ-supermajority of the last round of the previous wave, let's call it the blue supermajority.

Then the blue supermajority and the finalising σ-supermajority of b, let's call it the green supermajority, must have a correct agent in their intersection. Namely, there is a correct agent $p \in P$ with a blue p-block that must observe a green p-block (which may actually be the same block if W and W' are consecutive waves), which in turn must observe a σ-supermajority of blocks that endorse b. Since b' observes the blue p-block that observes the green p-block that ratifies b, then b is ratified by b'. ◀

▶ **Proposition 19** (τ Monotonicity). *τ is monotonic wrt $\supset$ over constitutionally-safe blocklaces.*

Proof. We have to show that if $B \subset B'$ for blocklaces B, B' then $\tau(B)$ is a prefix of $\tau(B')$.

By Observation 7, all the blocks final in B are also final in B'. By Observation 15, each wave in B, B' includes at most one ratified or final block, hence the ratified and final blocks are totally ordered by their depth. If B, B' have the same most-recent final block, or if B has no final block, then the proposition holds trivially. Assume $b \neq b'$ are the most-recent final blocks in B, B', respectively, so necessarily b' is more recent than b. Let $b = b_1, \ldots, b_k = b'$, $k \geq 2$, be the sequence of all ratified blocks in B' between and including b and b'. By Proposition 18 and the definition of τ, $\tau(b')$ will eventually call recursively $\tau(b)$ and hence $\tau(b)$ is a prefix of $\tau(b')$, implying that $\tau(B)$ is a prefix of $\tau(B')$. ◀

▶ **Observation 20.** *If the union $B \cup B'$ of two blocklaces is constitutionally-safe then $\tau(B)$ and $\tau(B')$ are consistent.*

Proof. By monotonicity of τ over constitutionally-safe blocklaces (Proposition 19), $\tau(B)$ and $\tau(B')$ are both prefixes of $\tau(B \cup B')$, which implies that one is a prefix of the other, namely they are consistent. ◀

Proof of Theorem 13. By assumption, the P-blocks in $B \cup B'$ include equivocations by at most $f = (2\sigma - 1)n$ agents, so this set is constitutionally-safe. Hence, by Observation 20, the outputs of any two correct agents running Constitutional Consensus are always consistent. By Observation 14, the protocol is safe. ◀

B.2 Liveness

Next, we discuss liveness. Having proved safety, we know that all output sequences of correct agents are subsequences of a common sequence. Let T be the minimal such sequence. Then the set of elements in T is the union of all the elements in output sequences of correct agents. Because all members of P participate in all epochs, Epoch Liveness requires that all their output sequences include all the transactions in T, which are all the transactions output by correct agents. Constitution Amendment Liveness trivially holds in the single-epoch case. We conclude with the following observation:

▶ **Observation 21** (Single-epoch Liveness). *A single-epoch consensus protocol is live if and only if the output sequences of all correct participants include all the transactions input or output by correct participants.*

▶ **Theorem 22.** *Single-epoch Constitutional Consensus is live.*

We first prove that Constitutional Consensus disseminates all correct agents' blocks and the blocks they depend on.

▶ **Proposition 23** (Dissemination). *If a block b is issued by a correct agent in Constitutional Consensus then $[b]$ is eventually received by all correct agents.*

Proof. By way of contradiction assume that $p, q \in P$ are correct, b_p is issued by p (and received by q since the network is reliable) and some block $b \in [b_p]$ is never received by q. Since by assumption b (and possibly other blocks in $[b_p]$) have not been received by q, then after receiving b_p, q issues a NACK-block b_q for which $b \in [b_q]$. Upon receipt of b_q by p, p will send (once) $[b_q]$, which includes b, to q. Since the network is reliable, q will eventually received b, a contradiction. ◀

The following observation follows directly from the definition of block validity:

▶ **Observation 24** (Sequential Advance). *If a blocklace B contains a round r block, then every round $0 < r' < r$ is advanced in B.*

We next show that the protocol does not get "stuck", i.e., agents continue to produce blocks when they are needed for progress. Because Constitutional Consensus is quiescent, agents do not have to generate infinitely many blocks. But if some agent issues a block in a round, that round eventually advances.

▶ **Proposition 25** (Advance). *If a correct agent p sends a block in some round r in Constitutional Consensus, then r eventually becomes advanced at every correct agent.*

Proof. Let p_1 be an agent that sends a block b in round r. Then when p_1 does so, $r-1$ is advanced in p_1's blocklace.

Assume by way of contradiction that r does not advance at some correct agent p_2. By Observation 24, no blocks of depth greater than r are added to p_2's blocklace. By Proposition 23, this means that no correct agent sends blocks in rounds $> r$.

Moreover, because r does not advance at p_2, it must be the case that p_2 does not accept a supermajority of blocks in round r, which by Proposition 23 implies that there is some correct agent p_3 that does not send a round r block.

By Proposition 23, p_3 receives all the blocks in $[b]$. Therefore, p_3 accepts all these blocks into its blocklace. Thus, round $r-1$ is advanced in p_3's blocklace.

In case r is a first round following a quiescent wave at p_3, r advances at p_3 upon receipt of b, and p_3 issues a round $r+1$ block, a contradiction.

Otherwise, r is the maximum advanced round at p_3 and is not a first round following a quiescent wave at p_3. Therefore, p_3 issues a round r block, a contradiction. ◀

We next prove liveness for finite runs.

▶ **Proposition 26** (Finite implies quiescence). *Consider a run of Constitutional Consensus in which the blocklace B of a correct agent p is finite in a suffix of the run. Then B is quiescent.*

Proof. Assume by way of contradiction that p's latest advanced round in the run, r, is not quiescent. Then, by the **Issue** rule, p eventually issues a block in round $r+1$ (at most 9Δ time after r is advanced).

◀

▶ **Proposition 27** (Block issuance). *If an agent p has payload to send, then p eventually issues a block with this payload.*

Proof. Assume by contradiction that p has a non-empty payload from some time when p's most advanced round is r_0 onward, and p never issues a block with this payload. First, consider the case that p's rounds do not advance in some suffix of the run. Then by Proposition 26, p's blocklace is quiescent. Hence, condition 1 of the **Issue** rule is satisfied, and the payload is sent. A contradiction. Therefore, p's advanced rounds increase indefinitely. Consider a time when p's highest advanced round $r > r_0$ and the payload has not been sent. Then p issues a block either in round $r + 1$ (by the **Issue** rule) or in round r (by the **Issue backlog** rule). A contradiction. ◀

▶ **Proposition 28** (Transaction Liveness in finite runs). *Consider a run of Constitutional Consensus in which the blocklace B of a correct agent is finite in a suffix of the run. Then all input transactions of correct agents are ordered in B.*

Proof. By Proposition 23, B includes all ordinary blocks sent by correct agents. By Proposition 26, B is quiescent. This means that its last wave includes a first round block b that is final in B, and all other blocks in the wave are empty. Because b is final, p calls $\tau(b)$. Also by assumption of B being quiescent, all non-empty ordinary blocks in B are in $[b]$, and hence ordered by $\tau(b)$. Moreover, by Proposition 27, no agents have payloads to send. It follows that all input transactions have been sent in ordinary blocks and ordered. ◀

▶ **Proposition 29** (Epoch Liveness in finite runs). *Consider a run of Constitutional Consensus in which the blocklace B of a correct agent is finite in a suffix of the run. Then all transactions output by correct agents are ordered in B.*

Proof. Consider a transaction t output by a correct agent p_1 and another correct agent p_2 whose blocklace B_2 is finite in a suffix of the run. Because t is at some point final in p_1's blocklace, its blocklace eventually includes a block b with t in its payload and a super-majority of blocks that observe b. This super-majority includes at least one p_3-block b' by a correct agent p_3. By Proposition 23, p_2 b' is included in B_2, and because $b \in [b]$, so is b. Because B_2 is finite in a suffix of the run, by Proposition 26, it is quiescent, which means that all non-empty blocks including b are final in B_2, as needed. ◀

Next, we show that after GST, all waves that have correct leaders in high throughput mode are finalising. Because quiescence is not monotonic, we consider here only waves k for which the predecessor wave, $k - 1$, is not quiescent at any correct agent at any time.

▶ **Proposition 30** (First round progress after GST). *Let r be a first round with a correct leader, let p be a correct agent at which round $r - 1$ advances at time t>GST, and assume that the wave that ends in round $r - 1$ is not quiescent in any correct agent's blocklace at any time. Then p sends a block in round r if and only if it is the round's leader. Moreover, p endorses a round r leader block in round $r + 1$.*

Proof. Let p_ℓ be the correct leader of round r. If $p = p_\ell$ then once $r - 1$ is advanced, p immediately issues a leader block for round r, causing round r to advance, and issues a block endorsing it in round $r + 1$.

Otherwise, p waits for a block from p_ℓ until time $t + 2\Delta$ and then sends p_ℓ an INFORM-block, which is received by $t + 3\Delta$, causing p_ℓ to send p a NACK-block and get the blocks it is missing by time $t + 5\Delta$. Because round $r - 1$ is advanced and its wave is not quiescent, p_ℓ sends a leader block, which is received by p by time $t + 6\Delta$. Some predecessors of this block might be missing, causing p to wait Δ and then send a NACK-block and get the missing blocks allowing it to accept the leader block by time $t + 9\Delta$.

When the leader block is received, the round advances and p's round $r+1$ block endorses the leader block. ◀

Below, we say that a round r *advances at time* t if there is some correct agent p such that round r is advanced in p's blocklace at time t and r is not advanced at any correct agent before time t.

▶ **Proposition 31** (Progress after GST). *Let r be a first round with a correct leader s.t. round $r-1$ advances after GST and the wave that ends in round $r-1$ is not quiescent in any correct agent's blocklace at any time. Then a round r leader block is finalized in its wave at all correct agents.*

Proof. By Proposition 30, the only correct agent that sends a round r block is the round's leader and all correct agents endorse this block in round $r+1$. This means that it is impossible to create a valid round $r+1$ block that does not endorse the leader, as such a block would have to point to a super-majority of round r blocks excluding the leader block. Therefore, every valid round $r+2$ block ratifies the leader block, and the block is finalized in round $r+2$ as soon as it advances at any correct agent. ◀

Next, we consider runs with infinitely many quiescent waves.

▶ **Proposition 32** (Infinite quiescent waves). *Consider a run of Constitutional Consensus in which infinitely many waves are quiescent in at least one correct agent's blocklace at some point in time. Then all the blocks of correct agents are ordered by all agents in this run.*

Proof. Let B be the (infinite) union of all blocklaces in the run. Let b_1 be a block of a correct agent in B. By Proposition 23, b_1 is eventually included in all correct agents' blocklaces. Therefore, it is approved by all correct agents' blocks from some wave onward. Because every quiescent wave is finalising, and finalization is monotonic, there are infinitely many finalising waves in B. Let b_2 be a final block that approves b_1 in some wave k. Once b_2 is final at some agent's blocklace, b_1 is ordered at that agent. We will show that b_2 is finalized at all agents.

Observe that since b_2 is finalized in wave k, it is approved by a super-majority in wave k. Because every valid first round block approves a super-majority in the previous round and there is at least one correct agent at the intersection of any two super-majorities, every valid first round block in any round $k' > k$ observes b_2.

Consider a finalising wave $k' > k$ in B and let b_3 be the first-round block that is finalized in wave k'. Then b_3 is endorsed by a super-majority in the second round. Because this super-majority includes correct agents, all correct agents eventually accept b_3 (by Proposition 23). If b_3 is not a leader block, then to be validly endorsed, $[b_3]$ must be quiescent, and so upon receiving b_3, all the correct agents learn that b_2 is finalized.

By the same token, if any non-leader first round block b is endorsed by a valid block of any agent in the second round of wave $k+1$, then $[b]$ is quiescent. This, in turn, means that every third round block in the wave either ratifies the leader block or observes a block whose closure is quiescent.

So when the third round of wave $k+1$ advances, every correct agent either observes a super-majority of blocks that ratify the leader block or observes a block whose closure is quiescent. In the former case the leader block is finalized, ordering all preceding ordinary blocks including b_2, and in the latter b_2 is already final in the block's closure. ◀

We are now ready to prove the liveness theorem.

Proof of Theorem 22. By Observation 21, it suffices to show that every input or output transaction of a correct agent is eventually included in the output sequence of every correct agent. By Proposition 27, all input transactions in agents' payloads are eventually sent in (ordinary) blocks. Similarly, every output transaction of a correct agent is included in some ordinary block. Let b be a non-empty ordinary p-block included in some correct agent's blocklace in a run of Constitutional Consensus. If p is correct, then by Proposition 23, every correct agent receives and incorporates b in its local blocklace. Otherwise, b is finalized in some blocklace, implying that it is approved by at least one q-block b' by a correct agent q. Again, by Proposition 23, every correct agent receives and incorporates every block in $[b']$, including b, in its local blocklace.

We consider three cases: First, assume the run is finite. By Propositions 28 and 29, all non-empty ordinary blocks sent or output by correct agents in this run are ordered at all correct agents, and the theorem holds. Second, consider a run in which infinitely many waves are quiescent in at least one correct agent's blocklace at some point in time. By Proposition 32, if p is correct then b is ordered, and otherwise b' is ordered, implying that b is ordered as well because $b \in [b']$.

Finally, assume the run is infinite and contains only finitely many waves that are quiescent in at least one correct agent's blocklace at some point in time. Let k be a wave whose last round advances after GST and after which no wave is quiescent at any correct agent's blocklace. By fairness of leader selection, there are inifinitely many waves $k' > k$ with a correct leader. By Proposition 31, a leader block is finalized at all correct agents in each of these waves, ordering all non-empty ordinary blocks from earlier rounds. ◀

C UDP-Ready Constitutional Consensus for Eventually-Reliable Networks

Grassroots platforms and protocols are geared for smartphone-based serverless implementation, and hence depend on smartphone-to-smartphone communication. Smartphones often reside behind NATs and Firewalls, which in turn are programmed with policies that make smartphones finding each other difficult. To overcome this, servers with known IP addresses (or domain names) running the STUN protocol help devices behind NATs find each other. However, after they do, establishing a direct TCP connection is difficult, and hence direct phone-to-phone communication typically employs UDP.

Constitutional Consensus (Algorithm 1) was specified assuming a reliable network. Here, we relax the assumption to eventually-reliable networks, and extend the protocol to be UDP-ready, by adding ACK-blocks and block resending. This protocol resends only ordinary blocks, as the reliable communication of dissemination-inducing blocks (NACK, ACK, and INFORM) before GST is not required. Note that the protocol is very different from the trivial solution of "implementing TCP upon UDP", as during correct execution the same block can arrive via multiple paths and blocks may arrive in many possible orders. The protocol does not need to impose an ordering on agent-to-agent block communication as done by a TCP link.

▶ **Definition 33** (ACK-block). *Given a block $b = (h, x, H)$, an* ACK-block *for b is the block $b' = (h', (ACK, h), \emptyset)$.*

The changes to make Constitutional Consensus UDP-ready are adding ACK to the Receive rule, adding resending to the NACK-rule, a new Resend rule, revising the notion of *judiciously send*, and adding a signature by the sender of an ordinary block, if different from its creator,

so that it can be responded to with an ACK-block, as detailed in Algorithm 3 below (additions to Algorithm 1 are highlighted). In Algorithm 3, $k \geq 2$ is a parameter chosen to taste. In principle, it could be added as a fourth parameter to the constitution and be amended similarly to Δ.

Algorithm 3 UDP-Ready Extension to Constitutional Consensus, code for agent p.

Prior to sending an ordinary non-p-block b, sign b.

Receive: Upon receipt of q-block b, if b is a NACK-block then judiciously send $[b]$ to q, else if b an INFORM-block such that $p = leader(depth(b))$ and $[b] \not\subseteq B \cup \{b\}$ then send once a NACK-block for b to q,
else if b is valid then add b to D and send an ACK-block for b to its sender.

Accept & Nack: If $[b] \subseteq B \cup \{b\}$ for some $b \in D$ then $B \leftarrow B \cup \{b\}$, $D \leftarrow D \setminus \{b\}$, else if a q-block b was added to D more than Δ time ago, and no NACK-block for b was sent during the last $k\Delta$, then send a NACK-block for b to q.

Resend: If $k\Delta$ has passed since the most-recent ordinary block b was sent to q, no ACK-block or NACK-block was received for b, and $b \notin [b_q]$ for any q-block $b_q \in B \cup D$, then resend b to q.

- *judiciously send* $[b]$ *to* q means send every $b' \in [b]$ to q unless (i) an ACK-block or a NACK-block was received for b', or (ii) $b' \in [b_q]$ for some q-block $b_q \in B \cup D$.

Mobile devices. Smartphones are mobile, and moving may result in changing their IP address. Hence, even if a phone-to-phone connection is established, it has to be re-established every time one of the phones changes their IP address. Blocklace dissemination can support mobile agents recovering each other's IP address via a joint stationary friend [69]. The problem is already present at protocols of lower-level than consensus such as the grassroots social graph [70] and the blocklace-based solution presented there [69] is applicable here as well.

Safety and liveness. The UDP-ready extension (Algorithm 3) to Constitutional Consensus only relates to dissemination-inducing blocks (an ACK-block and its handing) and does not affect ordinary blocks. Hence the definitions, propositions, and proofs regarding the safety of Constitutional Consensus—all stated in terms of ordinary blocks only—remain valid verbatim for the UDP-ready extension, and therefore its safety is a corollary of Theorem 13.

Regarding liveness, prior to GST ordinary blocks issued by correct agents are resent to correct agents until received. Thus, there is a time $t \geq$ GST by which every block sent by a correct agent priot to GST either has been received, or has been resent after GST but before t. Thus, any message sent before t would be received by $t + \delta$. After GST, the two models under consideration are the same. Thus, this time t in the UDP-ready protocol has the same properties as GST in the Constitutional Consensus protocol: Any message between correct agents, if sent before t arrives by $t + \delta$, and if sent after t arrives within δ. Hence, the liveness proofs of Constitutional Consensus hold for the UDP-ready protocol, replacing GST by t thus defined, with the liveness of the UDP-ready protocol being a corollary of Theorem 22.

D Constitutional Amendment Pseudocode

Epoch transition walkthrough. Algorithm 2 shows the additional logic supporting amendments in Algorithm 1. Every epoch commences with a constitutional genesis block that specifies its prevailing constitution.

Amendement finalisation: If, during the epoch, members of the prevailing population produce a valid decision d to amend the prevailing constitution, they endeavour to finalise this decision by producing only constitutional blocks with d as payload.

End Epoch: Once an agent finalises a constitutional block with d, it produces a CORONATE block with payload $(\text{CORONATE}, d)$, sends it to both the old and new populations of d, and ceases participation in the epoch. Members of the old population that receive it and have yet to finalise d can NACK any missing dependent blocks. After ceasing to participate in the old epoch, an agent no longer adds blocks from that epoch to its blocklace. However, it must continue to respond to **Belated nack** requests from agents who are missing blocks in order to finalise the old epoch (due to Byzantine equivocation). An old epoch may be garbage-collected upon learning that all agents have coronated an ensuing epoch.

Start Epoch: A member of the new population that receives a σ-supermajority of CORONATE blocks may commence the new epoch with d as the constitutional genesis block. An agent that is a member of both the new and old populations commences the new epoch after both receiving a σ-supermajority of CORONATE blocks and sending one of its own.

The algorithm's output includes *constitutional blocks* with constitutional amendment decisions as payload. When τ orders a constitutional block with amendment d for the first time, the protocol outputs the new constitution, suppressing subsequent copies.

E Safety and Liveness of Constitutional Amendments

▶ **Theorem 34** (Constitutional Amendment Correctness). *The Constitutional Consensus Algorithm (Algorithm 1) with Constitutional Amendments (Algorithm 2) is correct.*

We need to prove that in every run of the protocol, there exists a sequence T s.t. the two safety properties and two liveness properties of Definition 3 hold w.r.t. T. Note that the ith epoch e_i is in fact d_i. We have proven that Algorithm 1 is correct in each epoch. Therefore, there is a sequence T^i so that the output sequences produced by all participants in epoch e_i are consistent and valid w.r.t. T^i.

Let T be the concatenation of all these sequences, $T := T^1 \cdot T^2 \cdots$. We next prove that the protocol satisfies safety and liveness properties w.r.t. T. The theorem will follow.

▶ **Lemma 35** (Constitutional Amendment Safety). *Every run of the Constitutional Consensus Algorithm (Algorithm 1) with Constitutional Amendments (Algorithm 2) satisfies safety w.r.t. T defined above.*

Proof. Validity immediately follows from the fact that agents participate only in epochs of which they are members. Consistency follows from the fact that agents participate in epochs sequentially, namely, they stop running the protocol in epoch i before starting epoch j. ◀

▶ **Lemma 36** (Constitutional Amendment Liveness). *Every run of the Constitutional Consensus Algorithm (Algorithm 1) with Constitutional Amendments (Algorithm 2) satisfies liveness w.r.t. T defined above.*

Proof. Consider an epoch e with prevailing constitution (P, σ, Δ), and the two liveness requirements. If e is the last epoch then Epoch Liveness follows from the liveness of the

single-epoch Algorithm 1, Theorem 22. Else assume e is not the last epoch, and therefore it must have ended with a coronation by a σ-supermajority among P, all observing a wave finalizing a constitutional block b with payload d.

Let $p \in P$ be a correct agent. We argue that p must eventually coronate d, and while doing so, call $\tau(b)$. As all correct participants in the epoch e end it by calling $\tau(b)$, all would output the same blocks for e. If p is among the σ-supermajority coronating d we are done.

Else there is at least one correct agent $q \in P$ that coronated d and sent to p a CORONATE block b' observing a wave W that finalizes b. Since the network is reliable, p receives b' but may be missing some blocks in $[b']$. Then p sends a NACK(b')-block to q, who responds (even after epoch e has ended, thanks to the Respond to Belated NACK rule) by sending to p the missing blocks in W and their predecessors, upon receipt of which p finalizes b, issues a (CORONATE, d) block, and outputs $\tau(b)$.

Considering Constitution Amendment Liveness, once a constitutional amendment d is known, all correct participants follow the high-throughput protocol and produce only blocks with payload d. Eventually a correct leader will finalize its block, following the same argument as in the proof of Theorem 22, upon which all correct participants will eventually issue (CORONATE, d) blocks. ◀

F Constitutional Consensus is Grassroots

We recall the formal definition of a grassroots protocol from reference [70], instantiate it informally (bypassing the formalities of the definition in [70]) to the Constitutional Consensus protocol, and then argue that the protocol is grassroots.

▶ **Definition 37** (Oblivious, Interactive, Grassroots). *A protocol $\mathcal{F}$ is:*

1. oblivious *if for every $\emptyset \subset P \subset P' \subseteq \Pi$, a run of the protocol over P can proceed just the same in the presence of agents in P'.*
2. interactive *if for every $\emptyset \subset P \subset P' \subseteq \Pi$ and every run of the protocol over P, the protocol can proceed from any configuration c in r to interact with members in $P' \setminus P$, namely produce a run that is not possible only over P.*
3. grassroots *if it is oblivious and interactive.*

Note that the definition is stated in terms of any two populations $\emptyset \subset P \subset P' \subseteq \Pi$.

Being oblivious, in the context of Constitutional Consensus, means that if the prevailing populations of the protocol during a run are always included in P, then the presence of additional agents hanging around, those in $P' \setminus P$, which do not participate in the protocol, should not interfere with the protocol or affect its behaviour. Clearly, this is the case with Constitutional Consensus, as its prevailing populations may always choose not to amend the constitution to include members outside P. Hence Constitutional Consensus satisfies the requirement of being oblivious.

Being interactive, in the context of Constitutional Consensus, means that if the prevailing populations of the protocol in some prefix of a run ending in a configuration c are included in a set P, then the protocol has a computation starting from c that interacts with members outside P in a meaningful way, namely the protocol has a computation from c that reaches a configuration c', in which members of $P' \setminus P$ are included in the prevailing population, and thus c' cannot be reached if the prevailing populations are all restricted to P. Clearly, members of the prevailing population P, starting from configuration c, may amend the constitution to include in P a new member $p' \in (P' \setminus P)$ and thus reach a configuration c' in which $p' \notin P$ is a member. Hence Constitutional Consensus satisfies the requirement of

being interactive. Together, these two properties imply that Constitutional Consensus is indeed grassroots. We thus conclude:

▶ **Theorem 38.** *Constitutional Consensus is grassroots.*

Note that the two requirements cannot even be expressed in the context of the basic (no constitutional amendments) Constitutional Consensus protocol, as the population P is given and fixed. Also note that Constitutional Consensus being interactive critically-depends on the ability of the prevailing population P to decide to add new members.

G Related Work

Constitutional Consensus combines three properties: (*i*) democratic one-person-one-vote governance, (*ii*) decentralised BFT consensus, and (*iii*) self-governing constitutional amendment within the protocol. We survey prior work across five areas, corresponding to the five aspects of the protocol described in the introduction, and find that every existing system lacks at least one of these properties.

G.1 Smart Contracts and Digital Social Contracts

Deployed smart contract bytecode on Ethereum is immutable; amendments are achieved through proxy schemes that separate mutable state from swappable logic, controlled by admin keys, multisigs, or token-weighted governance [26]. Tezos [33] embeds self-amendment at the protocol level, and the amendment mechanism is itself subject to amendment. In both cases, governance is plutocratic and amendments apply to code, not to a broader protocol constitution.

A digital social contract [20] differs architecturally: it is a voluntary agreement among people, executed on the participants' own devices, and enforced by the participants themselves. Constitutional Consensus is designed for such contracts. Governance is democratic (one person, one vote) rather than plutocratic (one token, one vote), and amendments apply to the protocol constitution—including the participant set, supermajority threshold, and timing—not merely to code. These distinctions are summarised in Table 1.

G.2 DAOs and Blockchain Governance

Decentralised Autonomous Organisations [35] are the prevailing method for participant-governed digital organisations. Several blockchains can modify their own protocol rules through on-chain stakeholder voting: Tezos [33] through a five-period procedure requiring an 80% supermajority; Polkadot [79] through forkless runtime upgrades via on-chain referenda; Cardano [23] through a tripartite structure of Constitutional Committee, Delegated Representatives, and Stake Pool Operators; Cosmos [42] through its governance module; and the Internet Computer [19] through its Network Nervous System. All are universally plutocratic:[4] voting power is proportional to staked tokens. This is not accidental; stake-weighting provides built-in Sybil resistance, whereas democratic voting requires an external mechanism to establish unique personhood.

DAO frameworks built on existing blockchains are similarly plutocratic. Aragon [24] defaults to ERC-20 token-weighted voting. Compound Governor [48] and OpenZeppelin

[4] Cardano is utilizing one-person-one-vote but only for its Constitutional Committee, which has essentially only veto power.

Table 1 Comparing Smart Contracts and Digital Social Contracts.

	Smart Contract [26]	**Digital Social Contract** [20]
Among:	Accounts	People
Executed by:	Third-party validators	The people's smartphones
Cost:	Gas	None
Consensus:	Always required	Only when required by the contract
Governance:	Plutocratic (token-weighted voting)	Democratic (one person, one vote)
Amended via:	Proxy schemes and redeployment; bytecode is immutable	Constitutional amendments
Amendment authority:	Token-weighted governance	Participants themselves via supermajority
Applications:	Decentralized Finance (DeFi) [64, 37], Decentralized Autonomous Organizations (DAOs) [29, 27], Non-Fungible Tokens (NFTs) [78]	Social graph and social networking [69], grassroots currencies [67, 49], democratic communities and their federation [76]

Governor [57] are built around ERC20Votes, tying governance power to token holdings. Snapshot [54] offers configurable voting strategies including an equal-weight option, but is purely off-chain and admin-controlled. Optimism's Citizens' House [58] is a notable exception: it implements one-person-one-vote governance for retroactive public goods funding. However, the Citizens' House relies on Ethereum's proof-of-stake consensus, and the Optimism Foundation retains significant authority over proposal execution.

Schneider [63] argues that the economic logic embedded in blockchain systems systematically constrains democratic possibilities. Siddarth et al. [73] identify the gap directly: "the lack of a robust notion of personhood ... has led to the development of plutocracies." Talmon [75] examines DAO governance from a computational social choice perspective. Buterin [17] critiques coin voting as creating plutocratic governance vulnerable to vote buying and proposes proof-of-personhood alternatives. No DAO or self-amending blockchain achieves democratic governance with its own BFT consensus and self-governing constitutional amendment.

Consortium blockchain platforms such as Hyperledger Fabric [3] and R3 Corda [13] implement organisational governance rather than democratic governance. In Fabric, configuration changes—including membership, block parameters, and governance policies—require signatures from a majority of organisation administrators, collected off-chain and submitted as configuration update transactions. Corda centralises governance further: a network operator controls certificate issuance and network parameters. While Fabric's hierarchical policy system bears structural resemblance to constitutional amendment (each policy specifies who can modify it), decision-making authority rests with designated administrators, not with individual participants exercising equal voting rights.

G.3 Reconfiguration of Consensus Protocols

Constitutional amendments generalise reconfiguration in two respects:

1. *Scope.* Reconfiguration refers to changing the set of active agents (servers, processes), whereas a constitution describes the entire consensus protocol, and in principle any aspect

of it may be amended.

2. *Authority.* Even in self-governing systems like Raft [56], reconfiguration is a mechanism available to whoever holds the leader role. Constitutional amendments, by contrast, are decided by the participants themselves through a process specified by the constitution.

In Constitutional Consensus, constitutional amendment preserves the protocol instance: the transaction history (the blocklace) carries over from one epoch to the next under the new constitution. Starting a new instance, by contrast, would discard all prior state. In this work, the amendable component of the constitution consists of (P, σ, Δ) (Definition 1), parameters which affect the safety, liveness, and performance of almost any consensus protocol for eventual synchrony.

Lamport's Part-Time Parliament [43] informally described reconfiguration as intrinsic: the legislators' decrees determine who constitutes the parliament. Lamport, Malkhi, and Zhou [46] later formalised the mechanism as Method Rα, yet noted the idea "still appears not to be well understood." No formal framework was given for *who decides* on membership changes or how such decisions are governed.

The reconfiguration literature has focused on *how* membership changes execute safely. Vertical Paxos [45] introduced an auxiliary configuration master that determines the acceptor set for each configuration. DynaStore [2] allows any client to propose reconfigurations but treats them as externally triggered. Spiegelman et al. [74] explicitly model an administrator invoking reconfiguration operations. SMART [53] handles crash-fault reconfiguration via an external coordinator. Raft [56] treats configuration changes as special log entries committed through joint consensus of both old and new configurations. In the BFT setting, BFT-SMaRt [9] provides the first practical BFT library with reconfiguration support, but relies on an external trusted third party for membership decisions. Duan and Zhang [28] showed that prior BFT reconfiguration approaches can compromise liveness, and introduced Dyno, the first formally analysed dynamic BFT protocol. Proof-of-Stake protocols achieve a form of self-reconfiguration, where validator sets change based on staking transactions [41], but governance is plutocratic.

ComChain [77] and the subsequent SMaRtChain [8] achieve BFT reconfiguration without a trusted third party: current participants run Byzantine consensus to decide on membership changes and employ a forgetting protocol that decouples permanent keys from per-view consensus keys to prevent removed nodes from creating forks. However, neither system provides a governance mechanism: the criteria for accepting new members are application-specific (e.g., proof-of-work, certification by an authority). Moreover, both are limited to membership changes and do not support amending protocol parameters or governance rules.

Across all of these, decision authority is either left unspecified, delegated to an external entity, determined by application-specific criteria, or determined plutocratically. The mechanism of reconfiguration is well-studied; the question of who decides—through democratic governance—is not.

G.4 BFT Consensus Protocols

PBFT [21] introduced the first practical algorithm for BFT state machine replication, tolerating up to $\lfloor (n-1)/3 \rfloor$ Byzantine faults. HotStuff [80] achieved linear communication complexity with responsiveness and became the basis for the Diem blockchain. Tendermint [15] brought BFT consensus to blockchains and underlies the Cosmos network.

A second line of work replaces the leader-based approach with DAG-based protocols. DAG-Rider [39] is the first asynchronous BFT Atomic Broadcast protocol achieving optimal

resilience, amortised communication complexity, and time complexity simultaneously. Narwhal and Tusk [25] separate reliable transaction dissemination from ordering, enabling high throughput. Bullshark [32] adds a practical low-latency synchronous fast path. Jolteon and Ditto [31] combine leader-based consensus with an asynchronous DAG fallback. Mysticeti [4] achieves the theoretical lower bound of three message delays for commit latency. Raikwar et al. [60] provide a systematisation of knowledge on DAG-based consensus.

Constitutional Consensus builds on two protocols from this line of work: Cordial Miners [40], a DAG-based BFT Atomic Broadcast protocol, and Morpheus [50], which adapts between quiescent and high-throughput modes. Neither combines quiescence with a fully block-embedded protocol; Constitutional Consensus does, and adds constitutional amendment and democratic governance above them. None of the BFT protocols surveyed here addresses governance or self-amendment: they provide the consensus mechanism but leave the question of who participates and how rules change to external processes.

G.5 Grassroots Protocols and Democratic Identity

Democratic digital governance platforms achieve one-person-one-vote participation but rely on centralised infrastructure. Decidim [5], used by over 500 institutions, runs as a client-server web application. Loomio [52] provides equal-weight deliberation and voting, but all data resides on centralised servers. CONSUL Democracy [62, 22], serving approximately 250 governments, authenticates users via national ID but operates as a centralised web application. LiquidFeedback [6] published a blockchain prototype with a swarm-behaviour-inspired consensus algorithm where node influence derives from the number of accredited persons trusting each node, and the roadmap envisions voting on application code. However, LiquidFeedback implements liquid democracy with transitive delegation rather than one-person-one-vote; its consensus is not characterised as BFT; and the self-amendment capability remained a roadmap item.

Proof-of-personhood (PoP) systems solve the Sybil resistance prerequisite for democratic governance. Ford [30] argues that digital personhood—inalienable participation rights built on proof-of-personhood mechanisms—is a necessary foundation for digital democracy. PoPCoin [10] uses pseudonym parties to enable equal-weight participation in ByzCoin consensus, achieving democratic identity with BFT consensus but not addressing self-governing amendment. Humanode [38] is the closest existing system to combining all three properties: it runs BFT consensus (BABE/GRANDPA on Substrate) with equal per-node weight, using biometric identity verification. However, its self-governance mechanism (Vortex DAO) operates as an off-chain simulation as of early 2026, with full authority transfer planned approximately four years after mainnet launch; Vortex is designed as a DAO overlay on the consensus protocol, not as constitutional amendment embedded within it. Idena [36] achieves equal-weight consensus via flip-test identity verification, but its consensus is not formally characterised as BFT, and protocol upgrades are controlled by the development team. Encointer [12] provides proof of personhood through physical key-signing ceremonies but operates as a Polkadot parachain, inheriting its consensus.

Because Constitutional Consensus requires no external authority, global ledger, or staked capital, multiple independent instances can form, operate, and grow concurrently—the grassroots property [68, 70]. This contrasts with Proof-of-Stake protocols like Ethereum 2.0 [18], whose security depends on the staked token having real-world value, creating a winner-take-all dynamic that precludes independent instances. Democratic platforms achieve one-person-one-vote but are centralised; PoP systems solve identity but remain building blocks. To the best of our knowledge, no existing system combines democratic one-person-one-vote

governance, decentralised BFT consensus, and self-governing constitutional amendment into a single protocol.